# Acknowledgements & Copyright Notice

**© 2026 Dr. Pandarasamy Arjunan, Indian Institute of Science, Bangalore. All rights reserved.**

These slides are distributed under a Creative Commons License. See Creative Commons Attribution-ShareAlike 2.0 License.

The author makes these slides available for educational purposes. They may not be used or distributed for commercial purposes. You may copy, use, or distribute these slides for educational purposes, provided you cite the author as the source.

**Credits:** This presentation includes slides and images from various books, research articles, and online sources, including:

- NVIDIA's Edge AI and Robotics Teaching Kit
- Harvard - TinyML Courseware and Machine Learning Systems
- MIT - TinyML and Efficient Deep Learning Computing
- Dive into Deep Learning and Luis Serrano's Academy and his books and videos
- Hands-on Machine Learning with Scikit-Learn, Keras and TensorFlow book
- IoT Fundamentals: Networking Technologies, Protocols, and Use Cases for the Internet of Things, 2017 by David Hanes
- Introduction to IoT, 2021 by Anandarup Mukherjee; *AI at the Edge* by Daniel Situnayake, Jenny Plunkett - O'Reilly

The author acknowledges and thanks the creators for their contributions. Images and content are used under fair use for educational purposes.

**Disclaimer**: All logos, images, and other trademarks are the property of their respective owners. If you believe that any content in this presentation violates copyright laws, please contact samy@iisc.ac.in to have it promptly removed or appropriately attributed.

Instruction Enhancement Program (IEP) on

# Edge AI

*Lecture 4: Efficient AI and Efficiency Metrics*

Pandarasamy Arjunan (Samy)
Assistant Professor
Robert Bosch Centre for Cyber Physical Systems
Indian Institute of Science, Bengaluru
samy@iisc.ac.in

Image source: https://leverx.com

# Lecture Plan

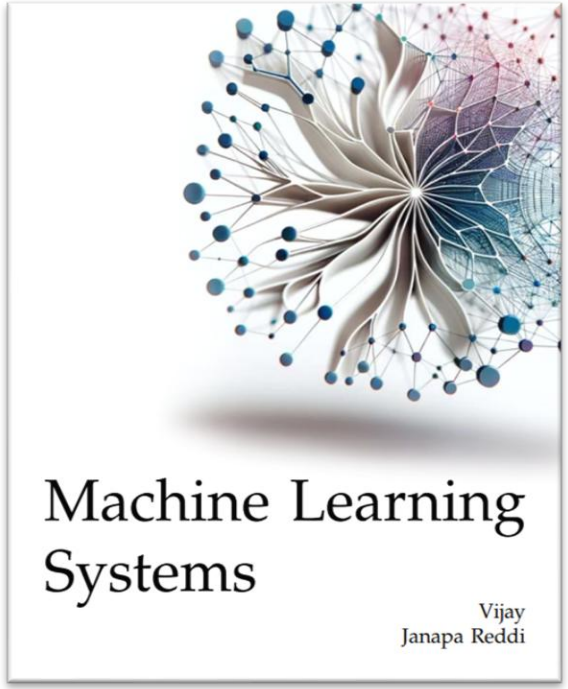
**Today**

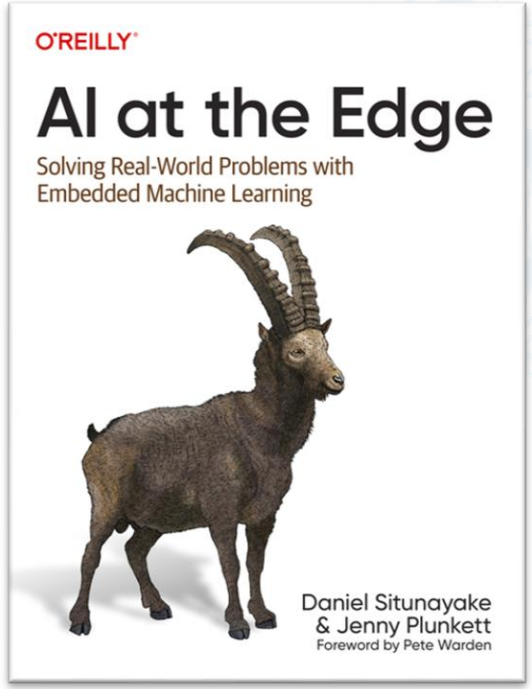
- **Lecture 1**: Introduction to IoT, Edge Analytics, and Edge AI
- **Lecture 2**: Tiny Machine Learning (TinyML)
- **Lab 1**: Human Activity Recognition using TinyML

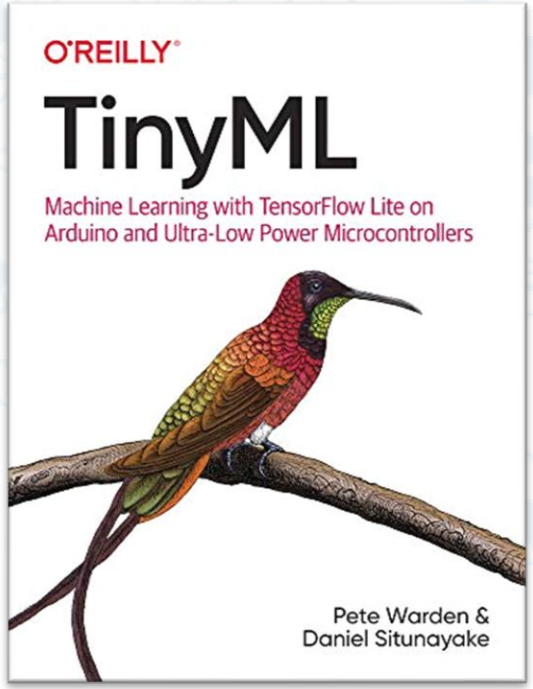
**Tomorrow**

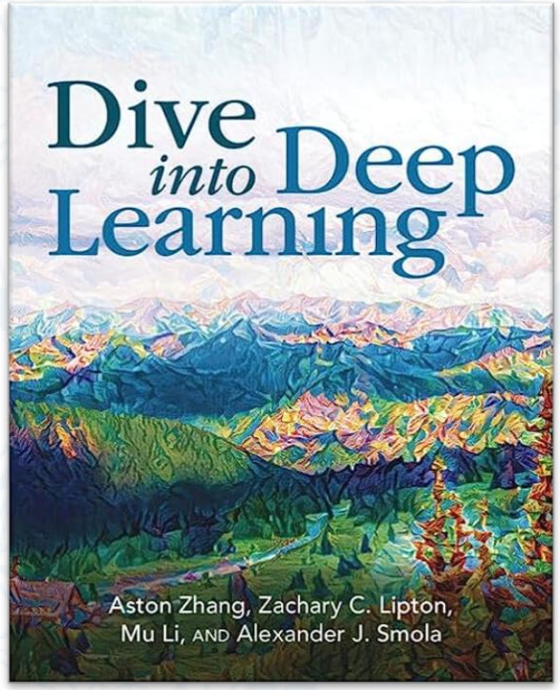
- **Lecture 3**: Overview of Embedded Computer Vision
- **Lecture 4: Efficient Deep Learning and Efficiency Metrics**
- **Lecture 5**: Model Optimization Techniques (Quantization, Pruning, etc.)
- **Lab 2**: Embedded Computer Vision
- **Lab 3**: Model Optimization for Edge AI (online)

# Books and reference materials



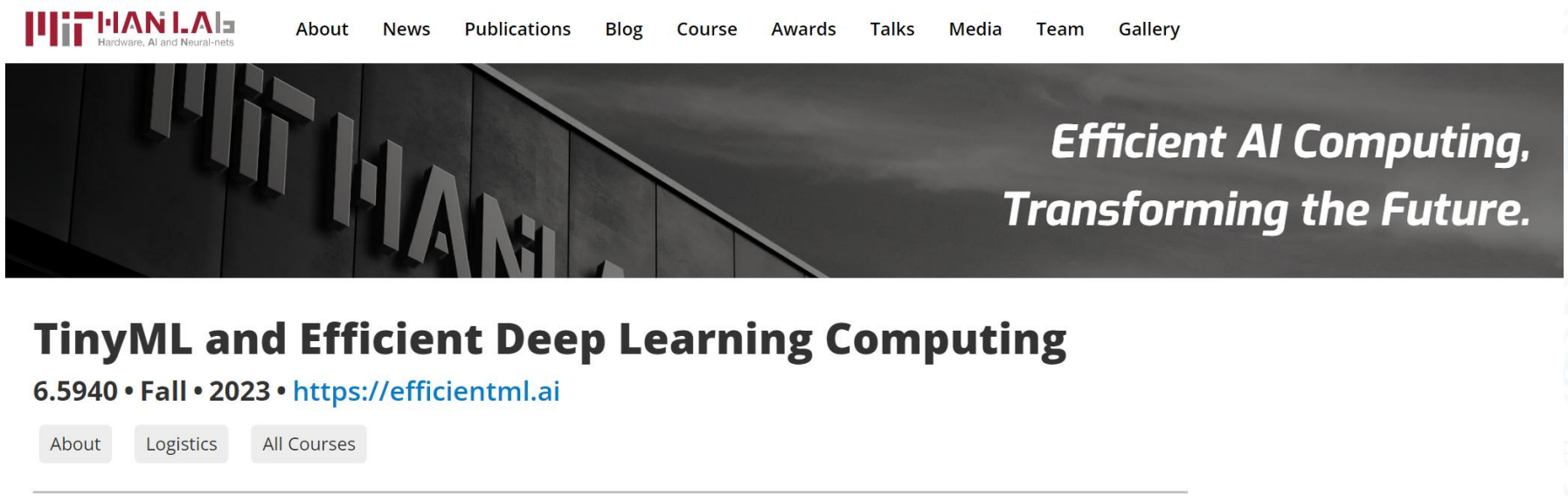



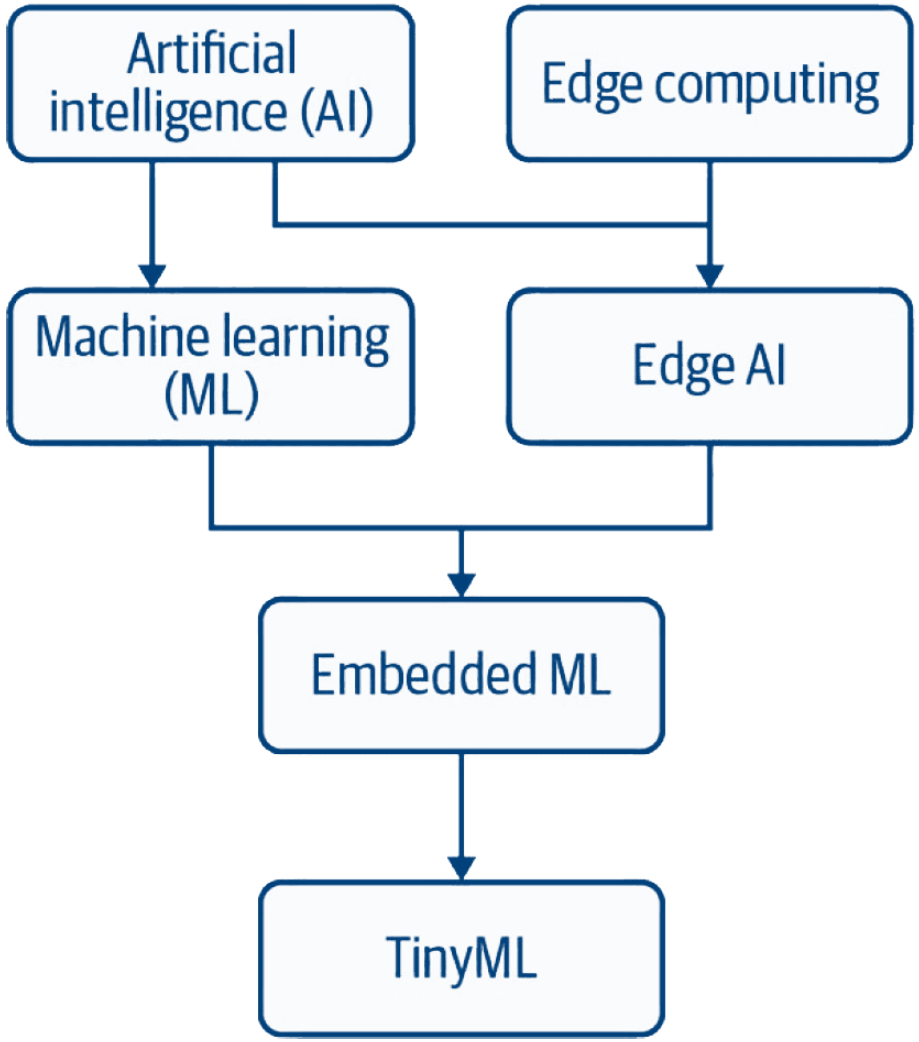

Plus, hands-on tutorials and research papers

# Reference

TinyML and Efficient Deep Learning Computing - https://hanlab.mit.edu/courses/2023-fall-65940



# The BIG picture of Edge AI




Processing data near the data source instead of sending it to the cloud.

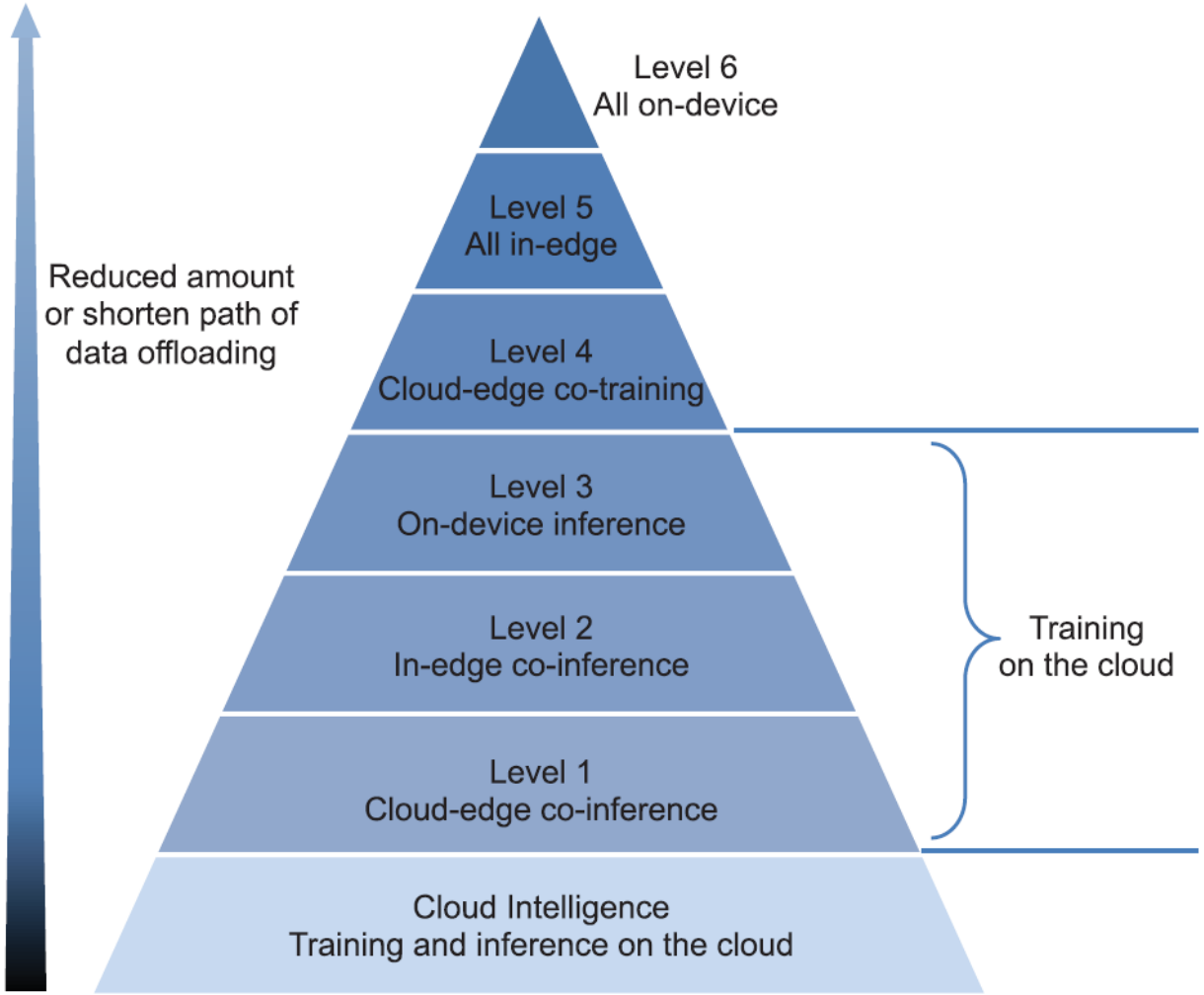
Executing and adapting AI models on edge devices, enabling local intelligence without reliance on the cloud.

Running ML models on resource-constrained embedded systems

Ultra-low-power ML on microcontrollers with extremely limited resources

Source: [3]

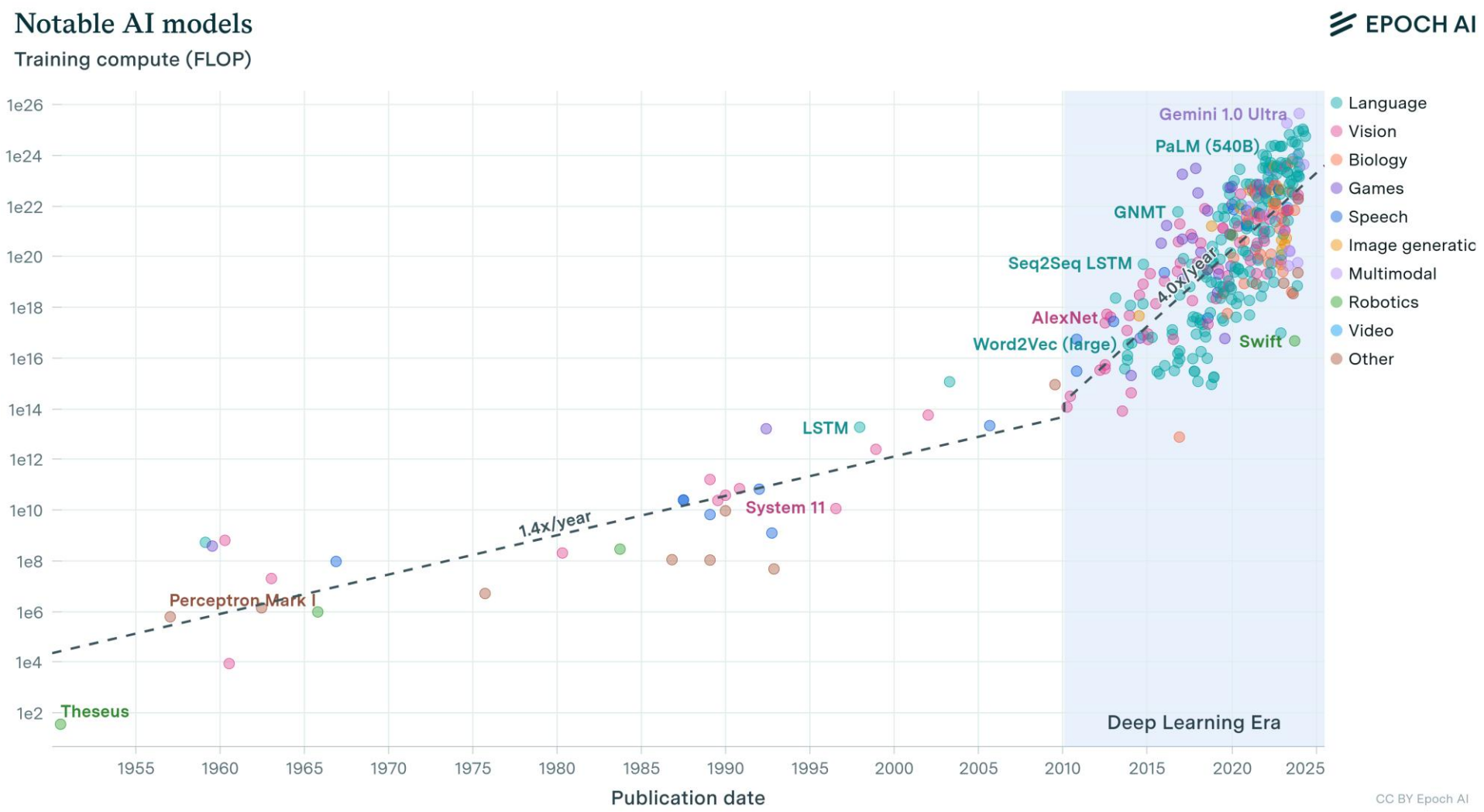
# Six-level of Edge Intelligence




Source: Edge Intelligence: Paving the Last Mile of Artificial Intelligence With Edge Computing (Zhou et al. 2019)

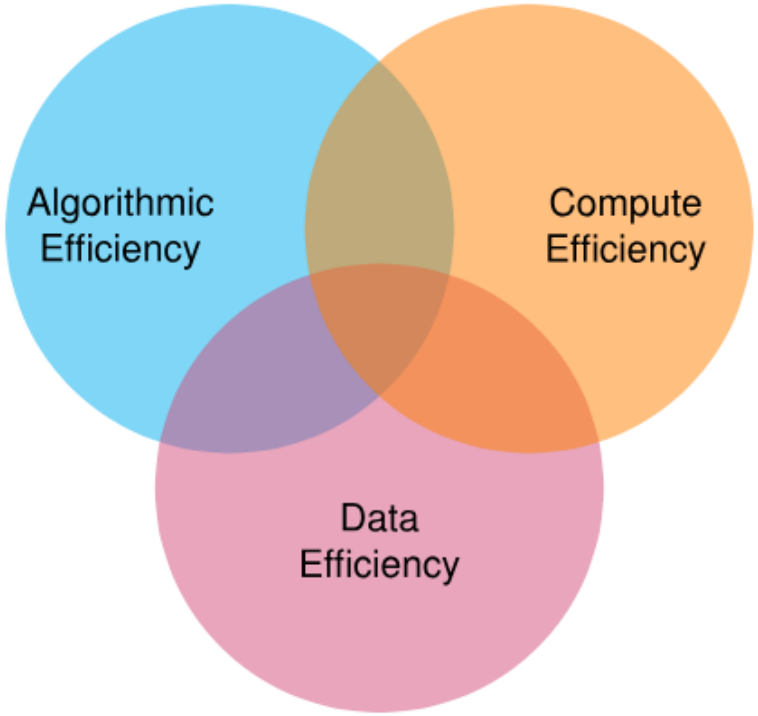
# Lecture 4: Efficient AI and Efficiency Metrics

# Model Training Compute Trends



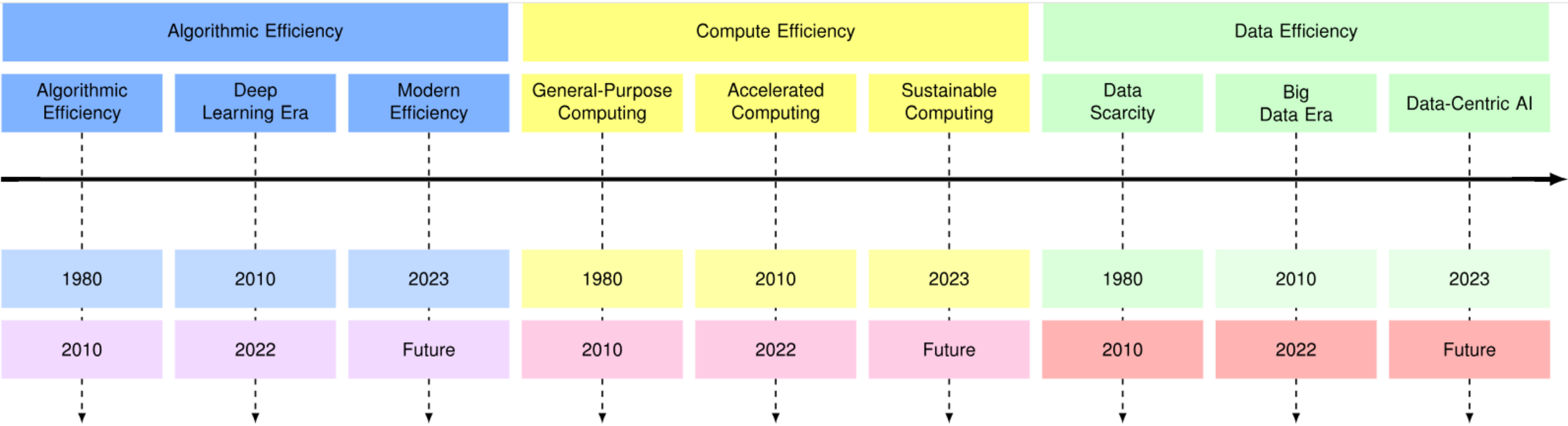
# Efficient AI

- **Algorithmic Efficiency** focuses on the model architecture.
- **Compute Efficiency** addresses hardware utilization
- **Data Efficiency** shapes how the model learns

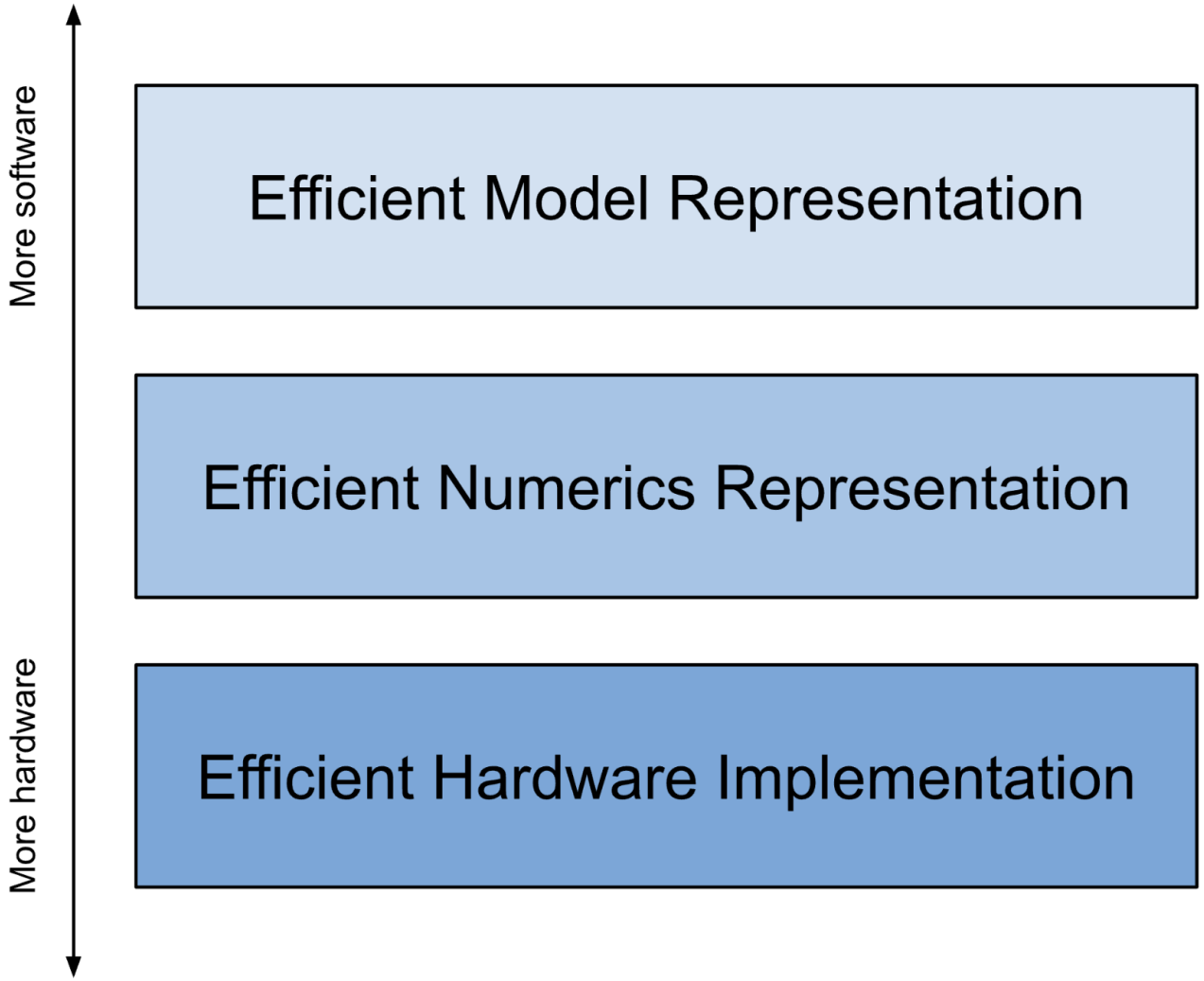


# Efficient AI

- **Algorithmic Efficiency** focuses on the model architecture.
- **Compute Efficiency** addresses hardware utilization
- **Data Efficiency** shapes how the model learns

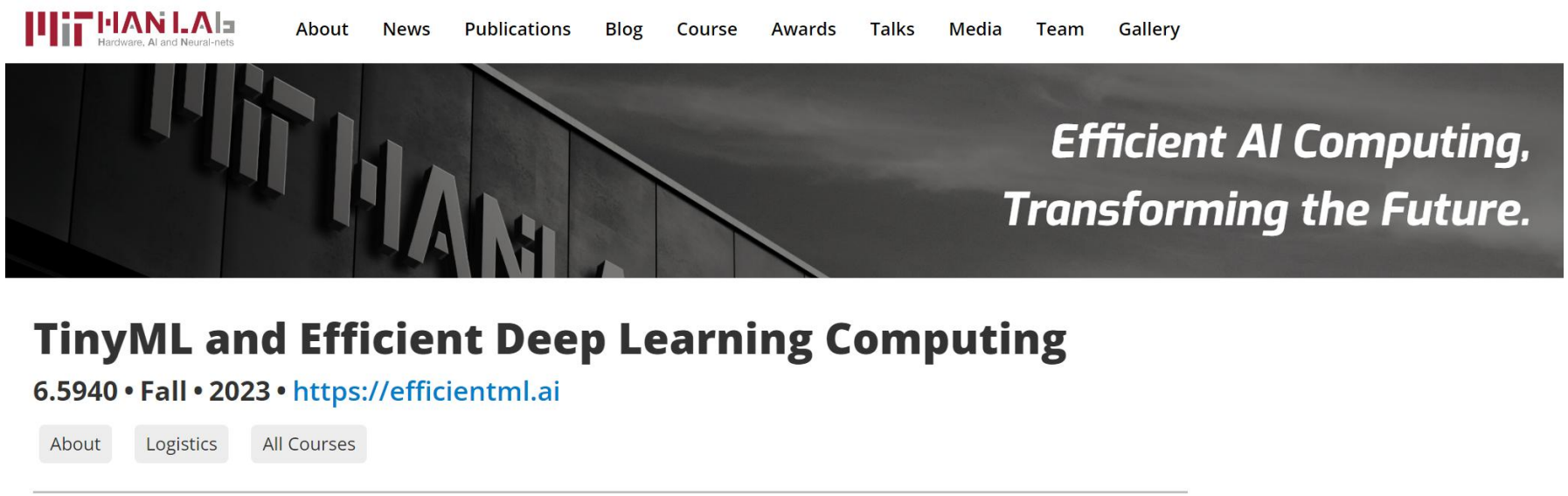


# Efficient AI



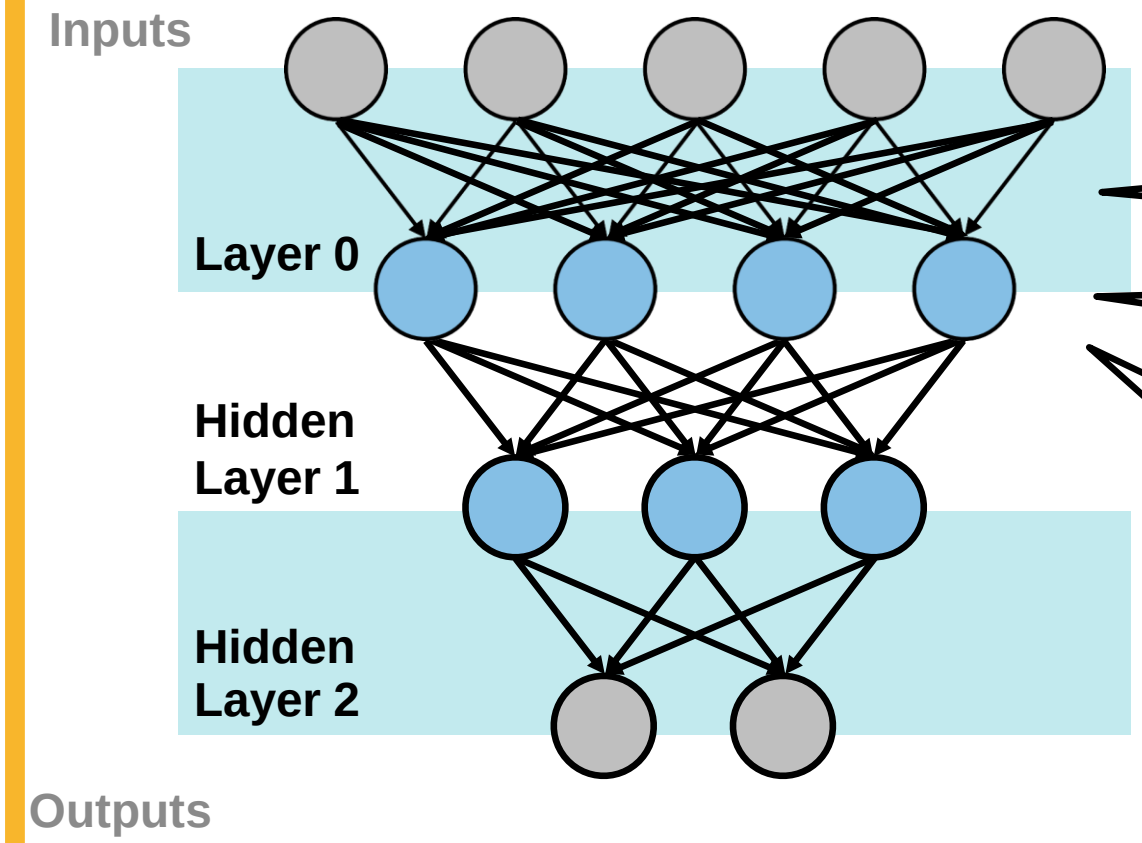
# Reference

TinyML and Efficient Deep Learning Computing - https://hanlab.mit.edu/courses/2023-fall-65940



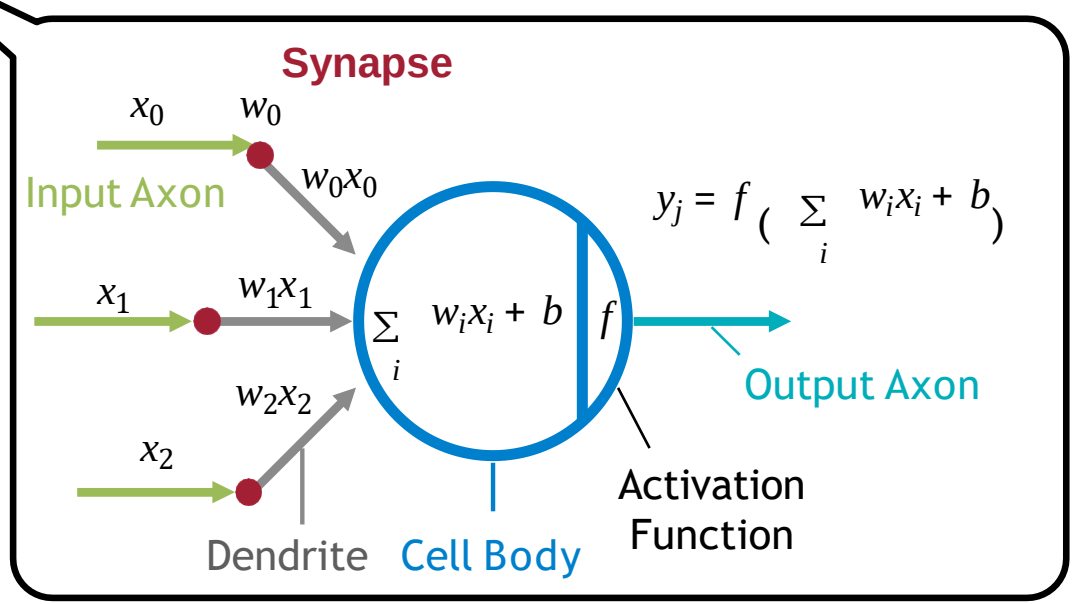
# Deep Neural Network

**3-Layer Neural Network**
**With 2 Hidden Layers**




**Synapses / Weights / Parameters**

**Neurons / Features / Activations**

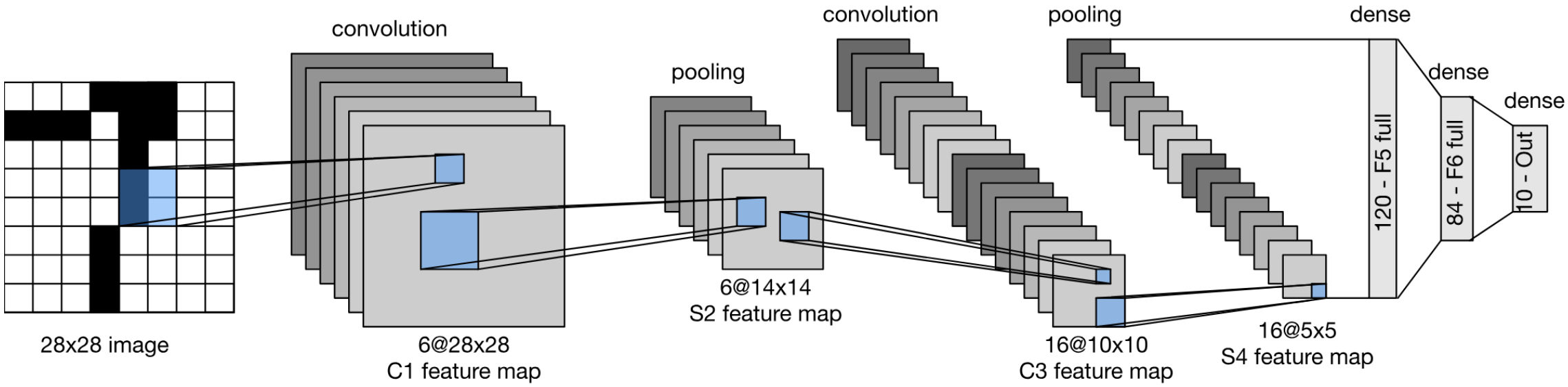



The dimensionality of these *hidden* layers determines the **width** of the model.

# LeNet

- The first published CNNs in 1998
- Recognizing handwritten digits in images
- Achieved an error rate of less than 1% per digit!

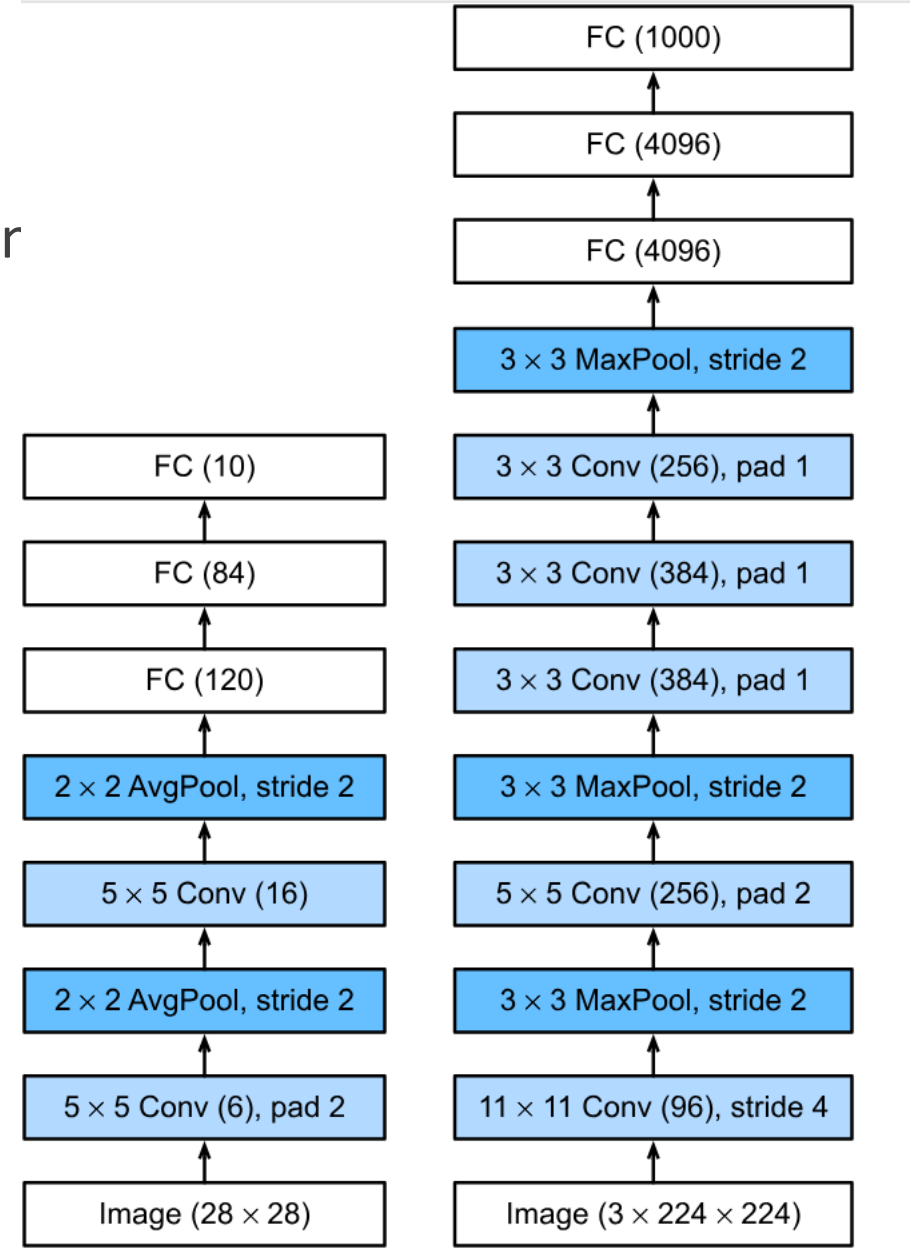
Expensive if we have many outputs



# AlexNet

- A major breakthrough - a deep CNN that could run or GPU
- Parallel computing

| | #parameters | | FLOP | |
|---|---|---|---|---|
| | **AlexNet** | **LeNet** | **AlexNet** | **LeNet** |
| **Conv1** | 35K | 150 | 101M | 1.2M |
| **Conv2** | 614K | 2.4K | 415M | 2.4M |
| **Conv3-5** | 3M | | 445M | |
| **Dense1** | 26M | 0.48M | 26M | 0.48M |
| **Dense2** | 16M | 0.1M | 16M | 0.1M |
| **Total** | 46M | 0.6M | 1G | 4M |
| **Increase** | 11x | 1x | 250x | 1x |



# Popular Neural Networks

| Model | Size (MB) | Top-1 Accuracy (%) | Top-5 Accuracy (%) | Parameters (M) | Depth | Inference Time (CPU) (ms) | Inference Time (GPU) (ms) |
|---|---|---|---|---|---|---|---|
| VGG16 | 528 | 71.3 | 89.8 | 138.4 | 23 | 721.2 | 444.2 |
| ResNet50 | 98 | 76.2 | 92.9 | 25.6 | 50 | 81.3 | 29.3 |
| InceptionV3 | 92 | 77.9 | 93.7 | 23.9 | 48 | 88.9 | 31.2 |
| MobileNetV2 | 14 | 71.6 | 90.3 | 3.5 | 53 | 47.8 | 0.78 |
| DenseNet121 | 33 | 74.9 | 92.2 | 8.1 | 121 | 62.3 | 23.5 |
| NASNetMobile | 23 | 74.1 | 91.8 | 5.3 | - | 56.4 | 20.1 |
| EfficientNetB0 | 29 | 77.1 | 93.3 | 5.3 | - | 55.3 | 19.9 |
| Xception | 88 | 79.0 | 94.5 | 22.9 | 71 | 95.4 | 33.7 |
| AlexNet | 233 | 56.5 | 79.1 | 61.0 | 8 | 114.3 | 42.9 |
| SqueezeNet | 4.8 | 57.5 | 80.3 | 1.2 | - | 43.6 | 15.8 |

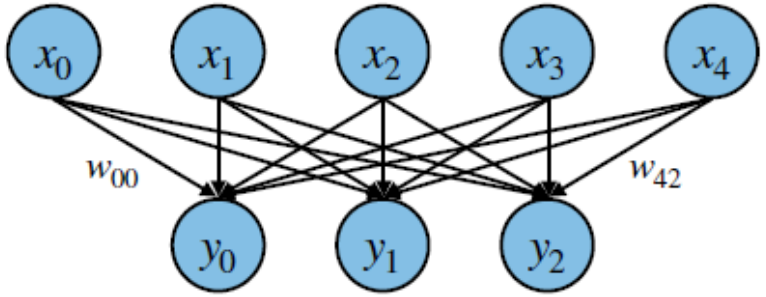
Parameters: The total number of trainable weights in the model.

# Fully Connected Layer

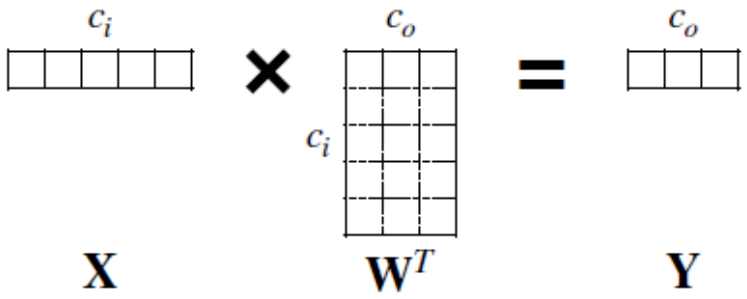
**The output neuron is connected to all input neurons.**

- **Shape of Tensors:**
  - Input Features $\mathbf{X}$ : $(1, c_i)$
  - Output Features $\mathbf{Y}$ : $(1, c_o)$
  - Weights $\mathbf{W}$ : $(c_o, c_i)$
  - Bias $\mathbf{b}$ : $(c_o, )$

| Notations | |
|---|---|
| | |
| $c_i$ | Input Channels |
| $c_o$ | Output Channels |




$$y_i = \sum_j w_{ij} x_j + b_i$$

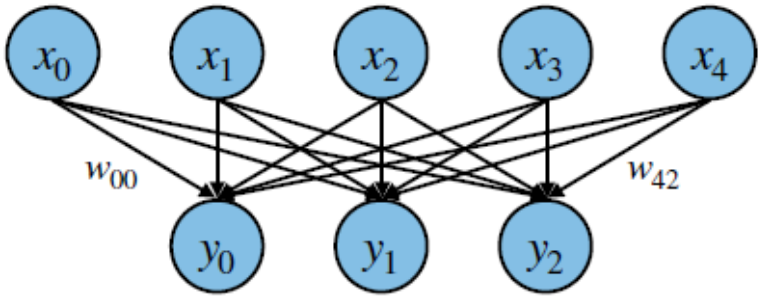


# Fully Connected Layer

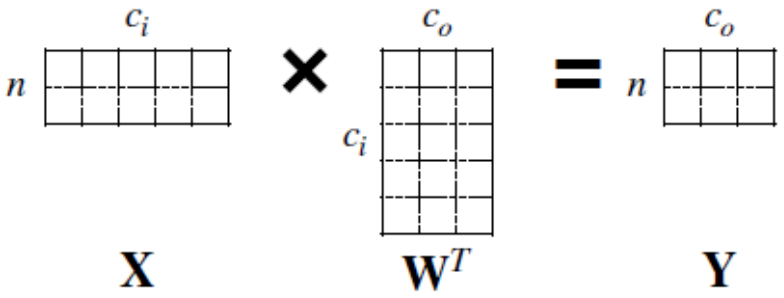
**The output neuron is connected to all input neurons.**

- **Shape of Tensors:**
  - Input Features $\mathbf{X}$ : $(n, c_i)$
  - Output Features $\mathbf{Y}$ : $(n, c_o)$
  - Weights $\mathbf{W}$ : $(c_o, c_i)$
  - Bias $\mathbf{b}$ : $(c_o, )$

| Notations | |
|---|---|
| $n$ | Batch Size |
| $c_i$ | Input Channels |
| $c_o$ | Output Channels |




$$y_i = \sum_j w_{ij} x_j + b_i$$

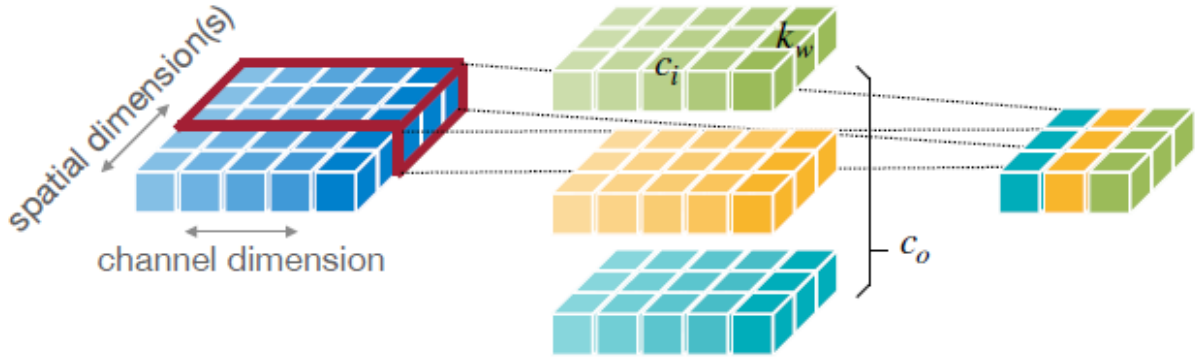


# Convolution Layer

**The output neuron is connected to input neurons in the receptive field.**

- **Shape of Tensors:** **1D Conv**
  - Input Features $\mathbf{X}$ : $\cancel{(n, c_i)}$ $(n, c_i, w_i)$
  - Output Features $\mathbf{Y}$ : $\cancel{(n, c_o)}$ $(n, c_o, w_o)$
  - Weights $\mathbf{W}$ : $\cancel{(c_o, c_i)}$ $(c_o, c_i, k_w)$
  - Bias $\mathbf{b}$ : $(c_o, )$

| Notations | |
|---|---|
| $n$ | Batch Size |
| $c_i$ | Input Channels |
| $c_o$ | Output Channels |
| $w_i, w_o$ | Input/Output Width |
| $k_w$ | Kernel Width |



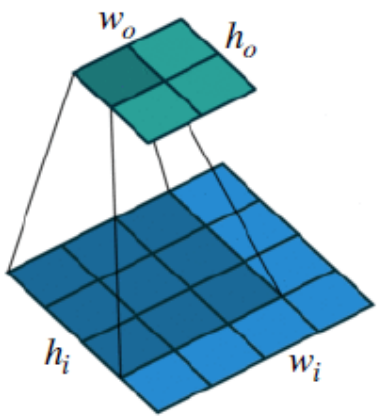

**Weight Sharing**

# Convolution Layer

**The output neuron is connected to input neurons in the receptive field.**

- **Shape of Tensors:**

| | 1D Conv | 2D Conv |
|---|---|---|
| Input Features $\mathbf{X}$ : $\cancel{(n, c_i)}$ | $(n, c_i, w_i)$ | $(n, c_i, h_i, w_i)$ |
| Output Features $\mathbf{Y}$ : $\cancel{(n, c_o)}$ | $(n, c_o, w_o)$ | $(n, c_o, h_o, w_o)$ |
| Weights $\mathbf{W}$ : $\cancel{(c_o, c_i)}$ | $(c_o, c_i, k_w)$ | $(c_o, c_i, k_h, k_w)$ |
| Bias $\mathbf{b}$ : $(c_o, )$ | | |




| Notations | |
|---|---|
| $n$ | Batch Size |
| $c_i$ | Input Channels |
| $c_o$ | Output Channels |
| $h_i, h_o$ | Input/Output Height |
| $w_i, w_o$ | Input/Output Width |
| $k_h$ | Kernel Height |
| $k_w$ | Kernel Width |

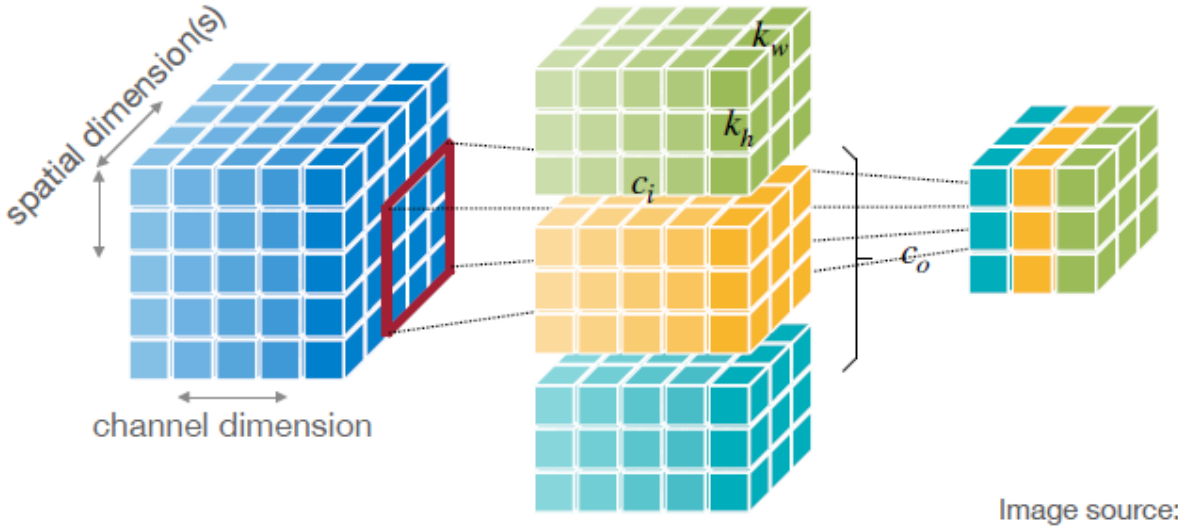


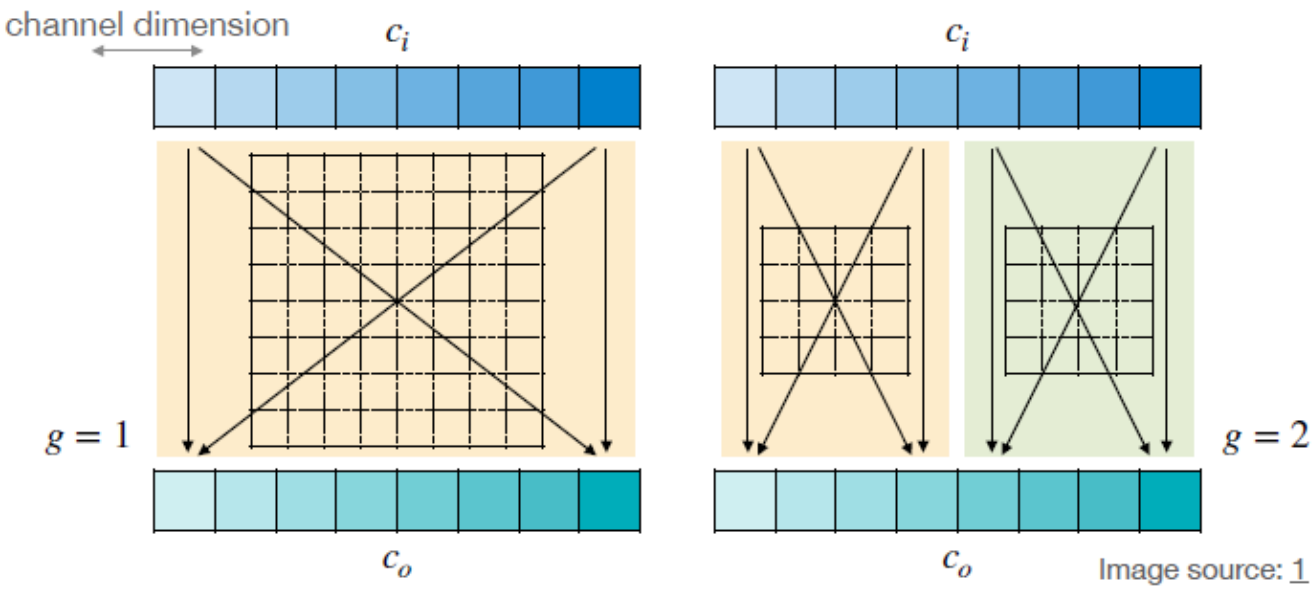

Image source: 1

# Grouped Convolution Layer

**A group of narrower convolutions**

- **Shape of Tensors:**
  - Input Features $\mathbf{X}$ : $(n, c_i, h_i, w_i)$
  - Output Features $\mathbf{Y}$ : $(n, c_o, h_o, w_o)$
  - Weights $\mathbf{W}$ : $\cancel{(c_o, c_i, k_h, k_w)}$ $(g \cdot c_o/g, c_i/g, k_h, k_w)$
  - Bias $\mathbf{b}$ : $(c_o, )$

| Notations | |
|---|---|
| $n$ | Batch Size |
| $c_i$ | Input Channels |
| $c_o$ | Output Channels |
| $h_i, h_o$ | Input/Output Height |
| $w_i, w_o$ | Input/Output Width |
| $k_h$ | Kernel Height |
| $k_w$ | Kernel Width |
| $g$ | Groups |

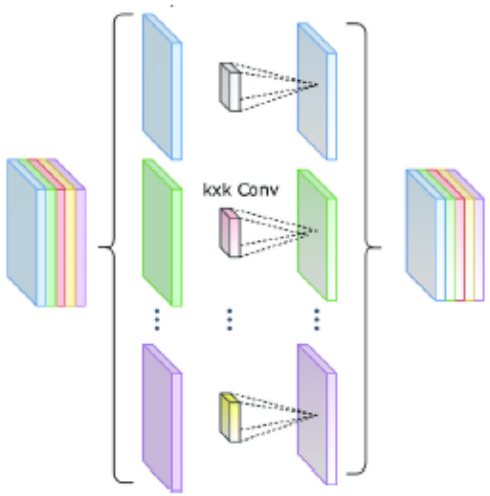


# Depthwise Convolution Layer

**Independent filter for each channel:** $g = c_i = c_o$ **in grouped convolution**

- **Shape of Tensors:**
  - Input Features $\mathbf{X}$ : $(n, c_i, h_i, w_i)$
  - Output Features $\mathbf{Y}$ : $(n, c_o, h_o, w_o)$
  - Weights $\mathbf{W}$ : $\cancel{(c_o, c_i, k_h, k_w)}$ $(c, k_h, k_w)$
  - Bias $\mathbf{b}$ : $(c_o, )$

| Notations | |
|---|---|
| $n$ | Batch Size |
| $c_i$ | Input Channels |
| $c_o$ | Output Channels |
| $h_i, h_o$ | Input/Output Height |
| $w_i, w_o$ | Input/Output Width |
| $k_h$ | Kernel Height |
| $k_w$ | Kernel Width |
| $g$ | Groups |



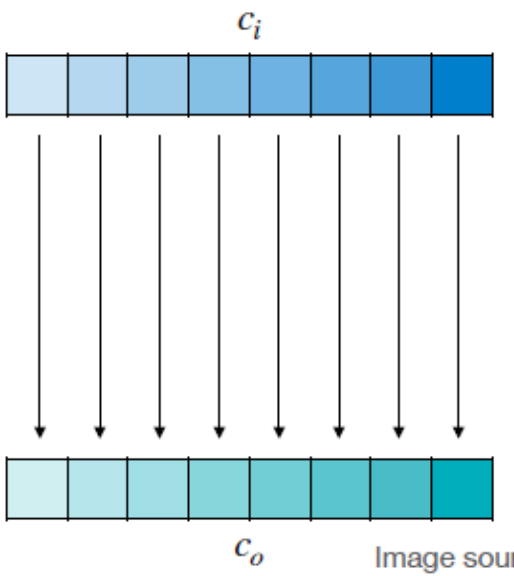
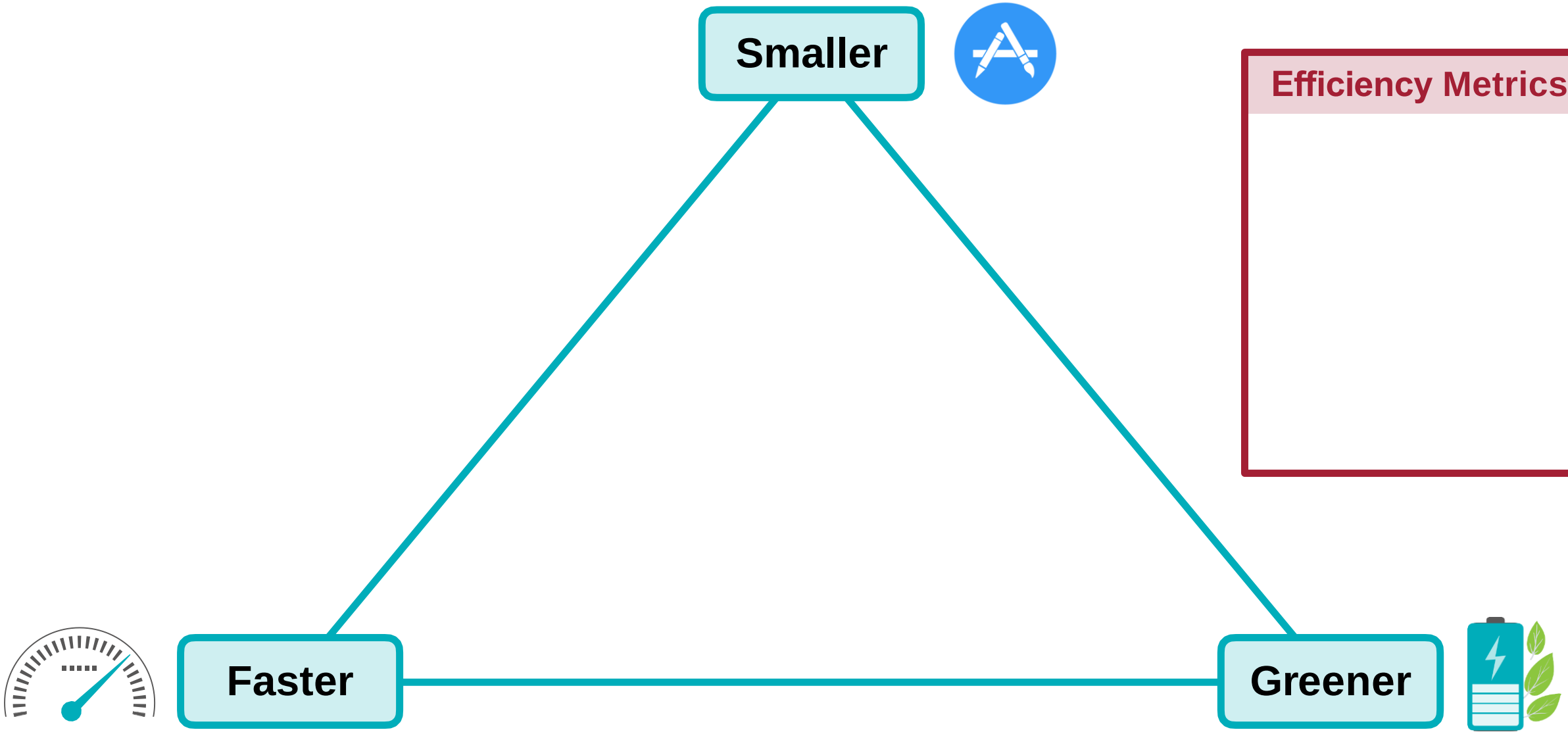




Image source: 1, 2

# Efficiency Metrics

# Efficiency Metrics




**Efficiency Metrics**

Image source: 1

# Efficiency Metrics

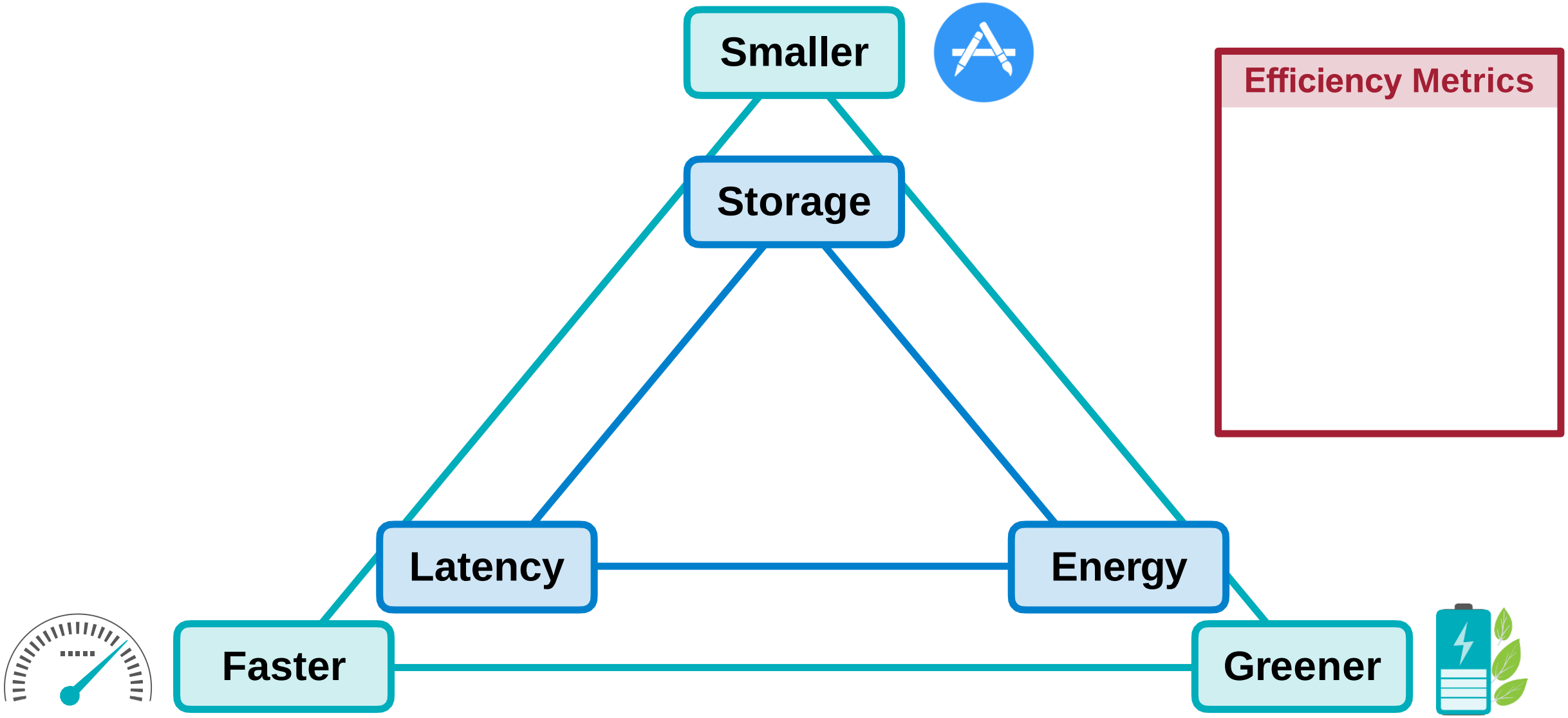
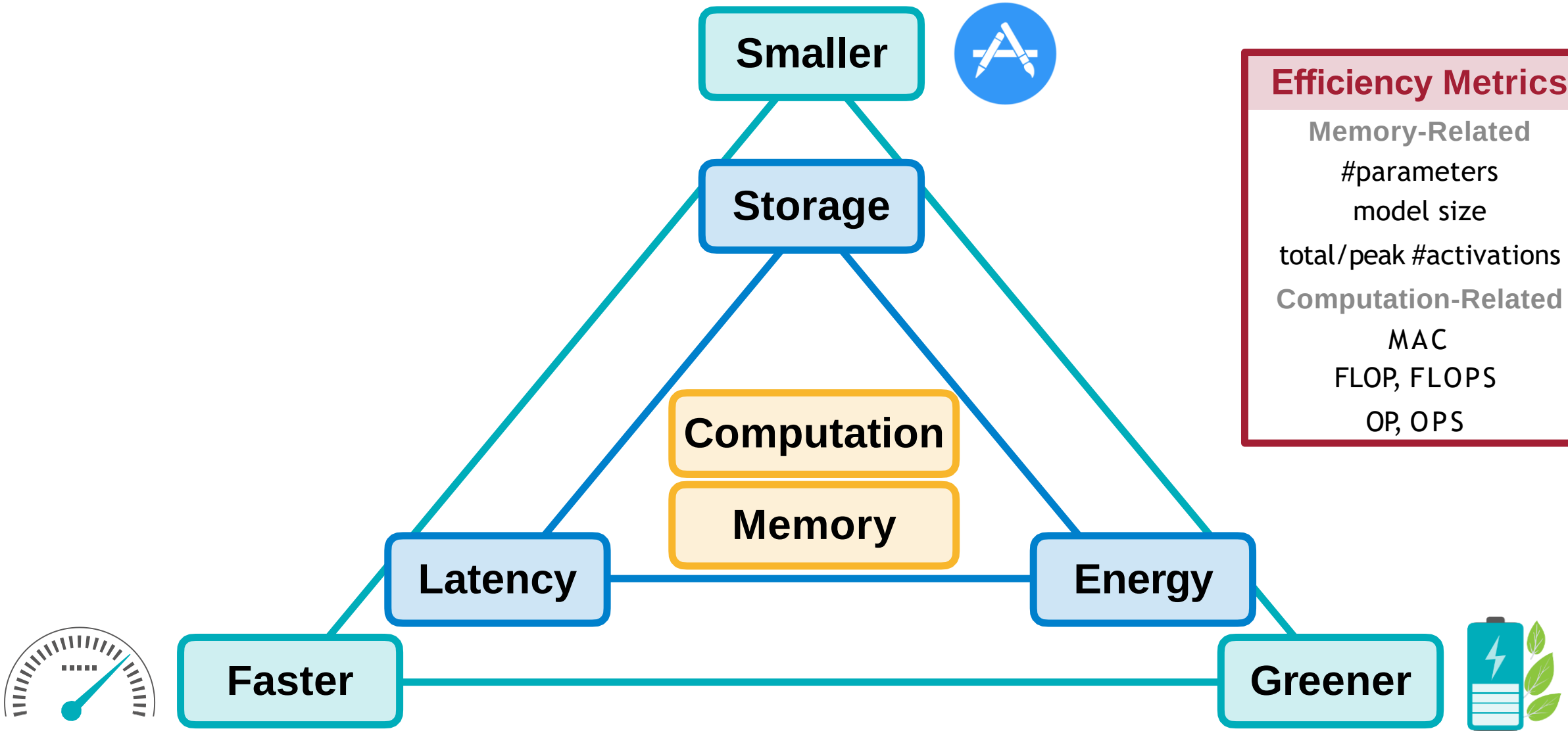



Image source: 1

# Efficiency Metrics




**Efficiency Metrics**

**Memory-Related**

#parameters

model size

total/peak #activations

**Computation-Related**

MAC

FLOP, FLOPS

OP, OPS

Image source: 1

# Latency

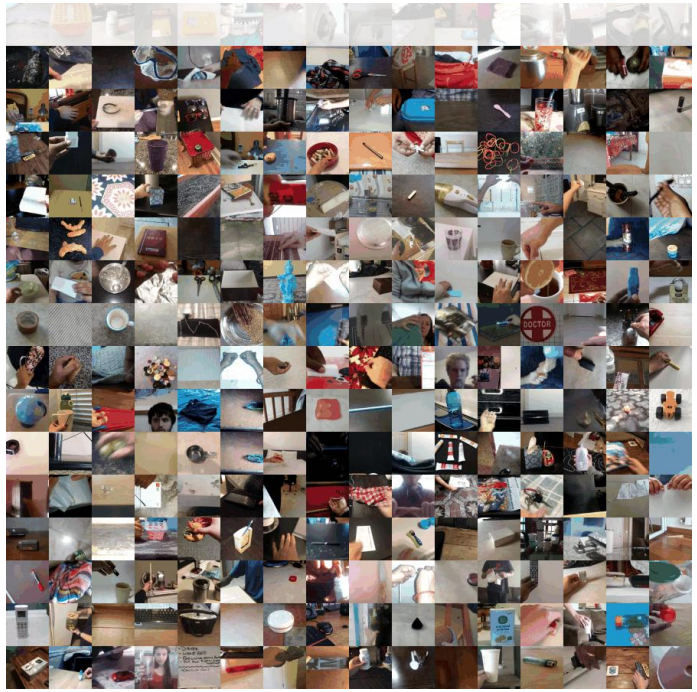
Measures the delay for a specific task

**High Latency**
**638ms**

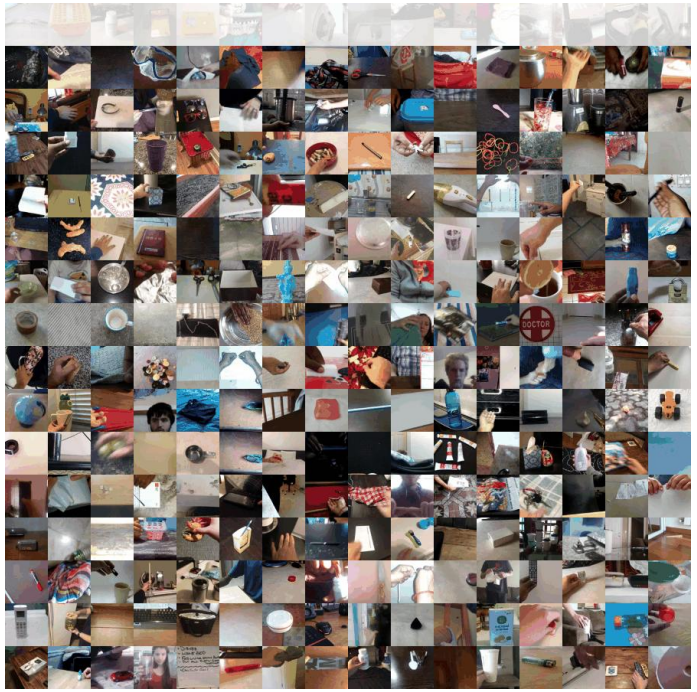
**Low Latency**
**46ms**

# Throughput

Measures the rate at which data is processed

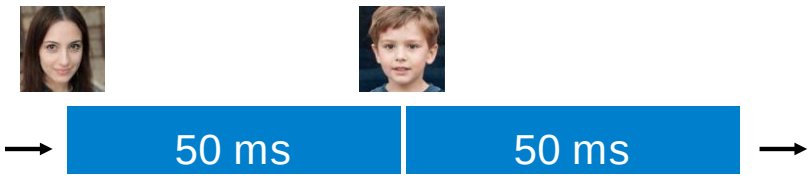


**Low Throughput = 6.1 video/s**



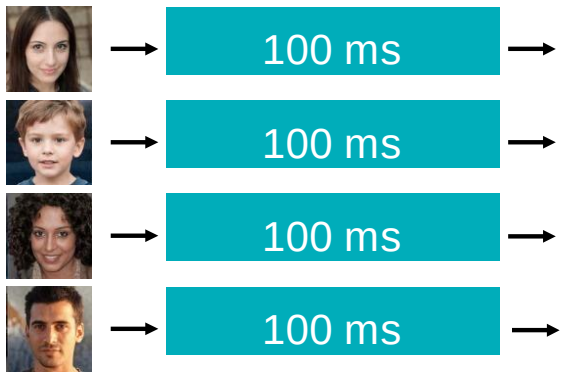
**High Throughput = 77.4 video/s**

# Latency vs. Throughput




Design 1

Latency: 50 ms
Throughput: 20 image/s




Design 2

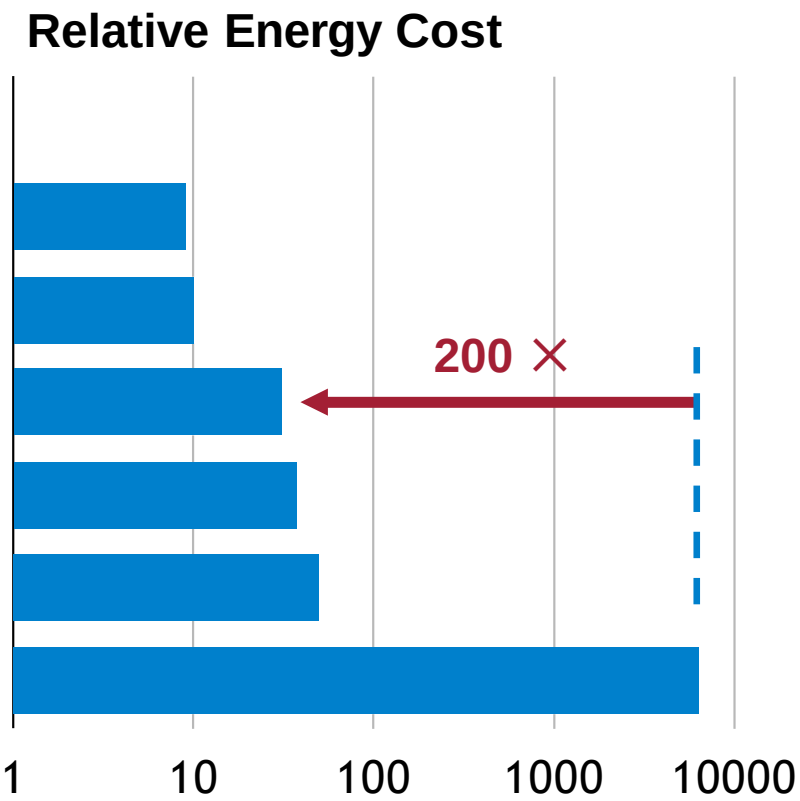
Latency: 100 ms
Throughput: 40 image/s

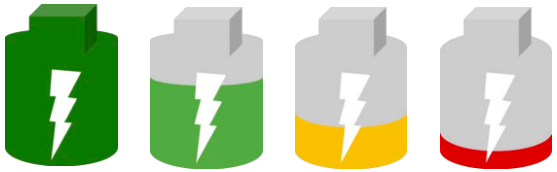
# Energy Consumption

Data movement → more memory reference → more energy

| Operation | Energy [pJ] |
|---|---|
| 32 bit int ADD | 0.1 |
| 32 bit float ADD | 0.9 |
| 32 bit Register File | 1 |
| 32 bit int MULT | 3.1 |
| 32 bit float MULT | 3.7 |
| 32 bit SRAM Cache | 5 |
| 32 bit DRAM Memory | 640 |

Rough Energy Cost For Various Operations in 45nm 0.9V

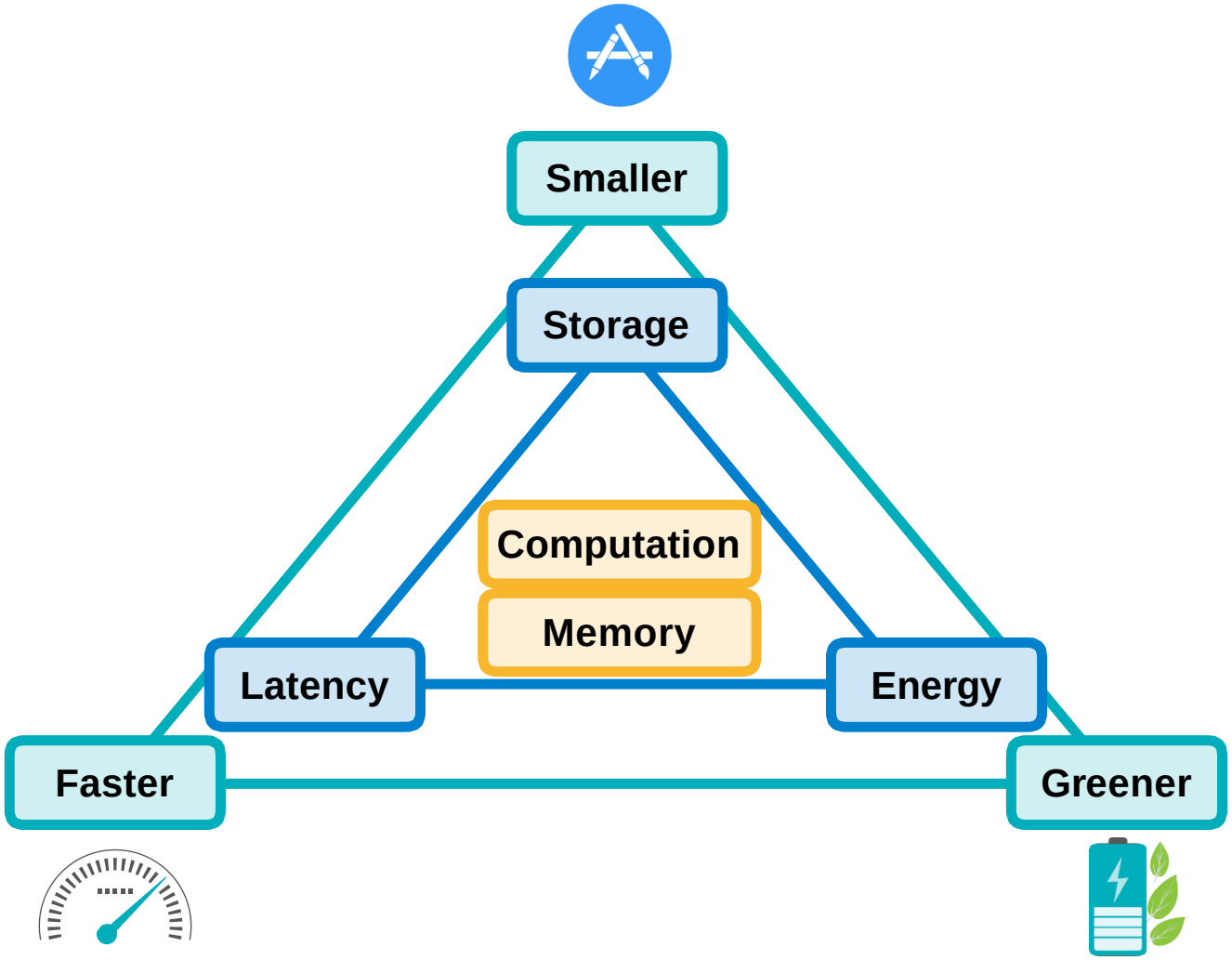


Battery images are in the public domain
Image 1, image 2, image 2, image 4

Computing's Energy Problem (and What We Can Do About it) [Horowitz, M., IEEE ISSCC 2014]

# Efficiency of Neural Networks




**Efficiency Metrics**

**Memory-Related**

#parameters

model size

total/peak #activations

**Computation-Related**

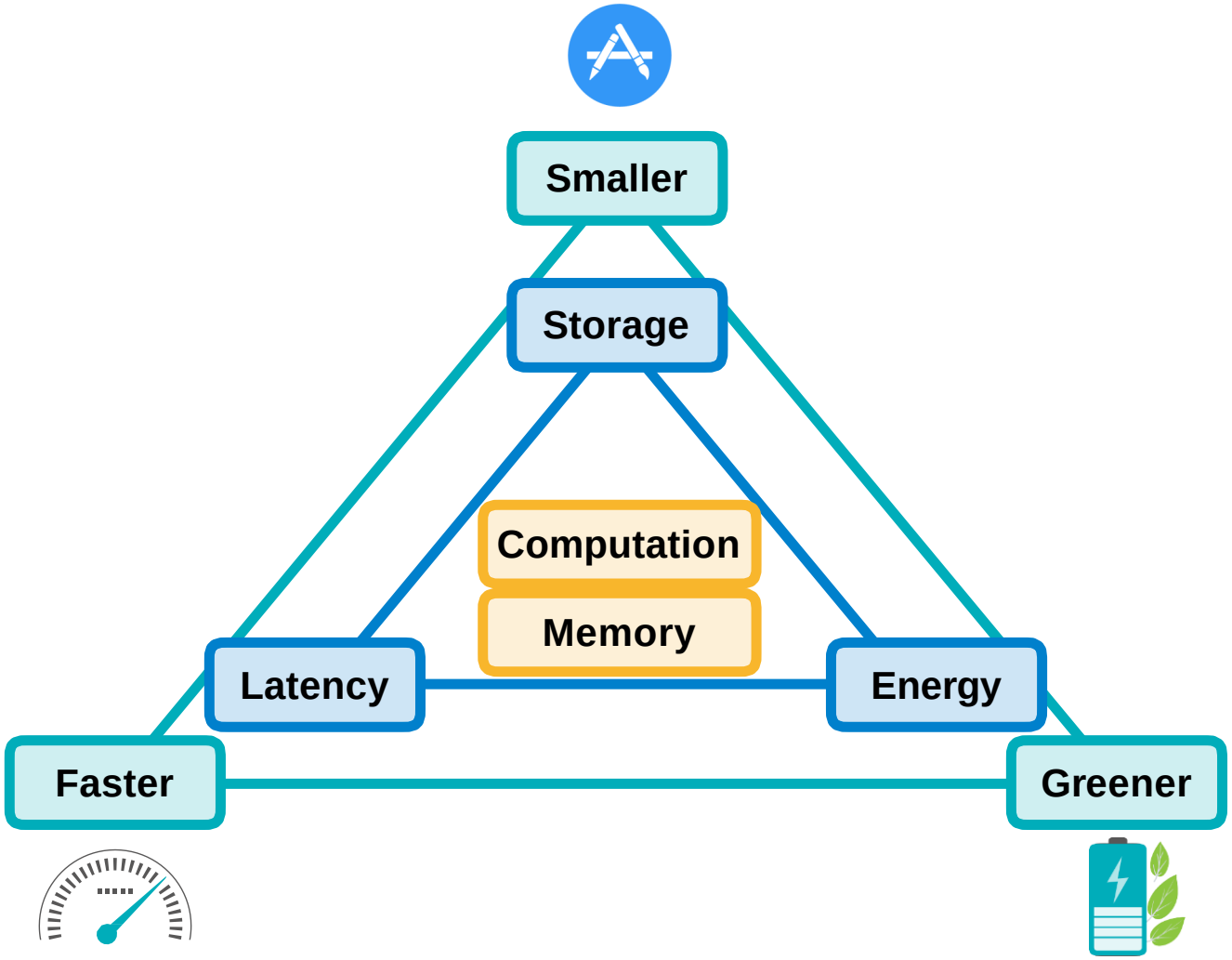
MAC

FLOP, FLOPS

OP, OPS

Image source: 1

# Efficiency of Neural Networks




**Efficiency Metrics**

Memory-Related

#parameters

model size

total/peak #activations

Computation-Related

MAC

FLOP, FLOPS

OP, OPS

Image source: 1

# Number of Parameters (#Parameters)

- #Parameters is the parameter (synapse/weight) count of the given neural network, *i.e.*, the number of elements in the weight tensors.

# Learnable Parameters (Trainable)

The total number of learnable parameters is the sum of all:

1. **Conv layer parameters** (weights + biases).
2. **FC layer parameters** (weights + biases).

# Learnable Parameters (Trainable)

These parameters are updated during training via backpropagation.

- **Weights (Filters/Kernels)** – The values in the Conv kernels that the model learns to detect patterns.

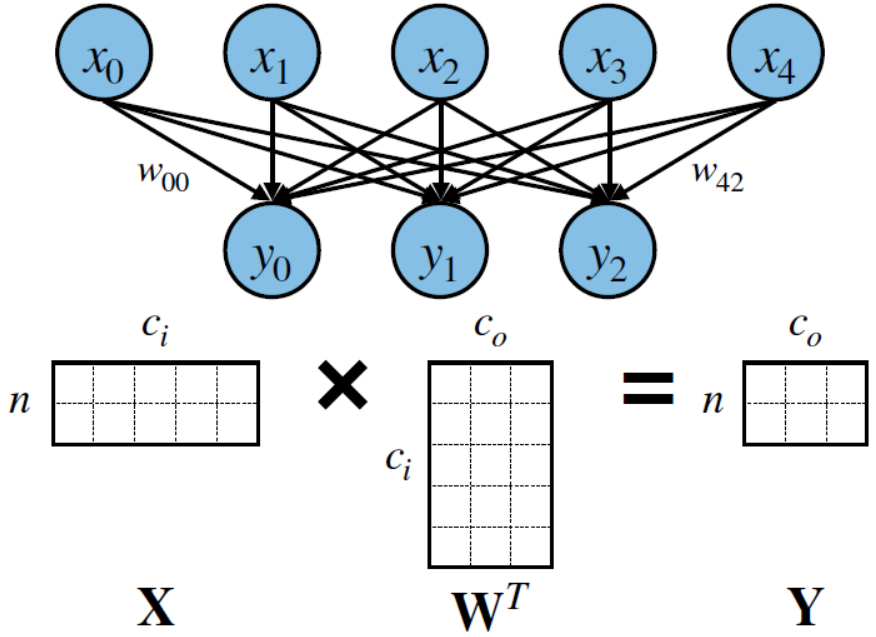
  If a Conv layer has **32 filters**, each of size **3×3×3 (RGB input)**, the number of learnable parameters is:

  $(3\times3\times3)\times32=864$ parameters

- **Biases** – Additional learned values added to the output of each neuron in Conv and FC layers.

# Number of Parameters (#Parameters)

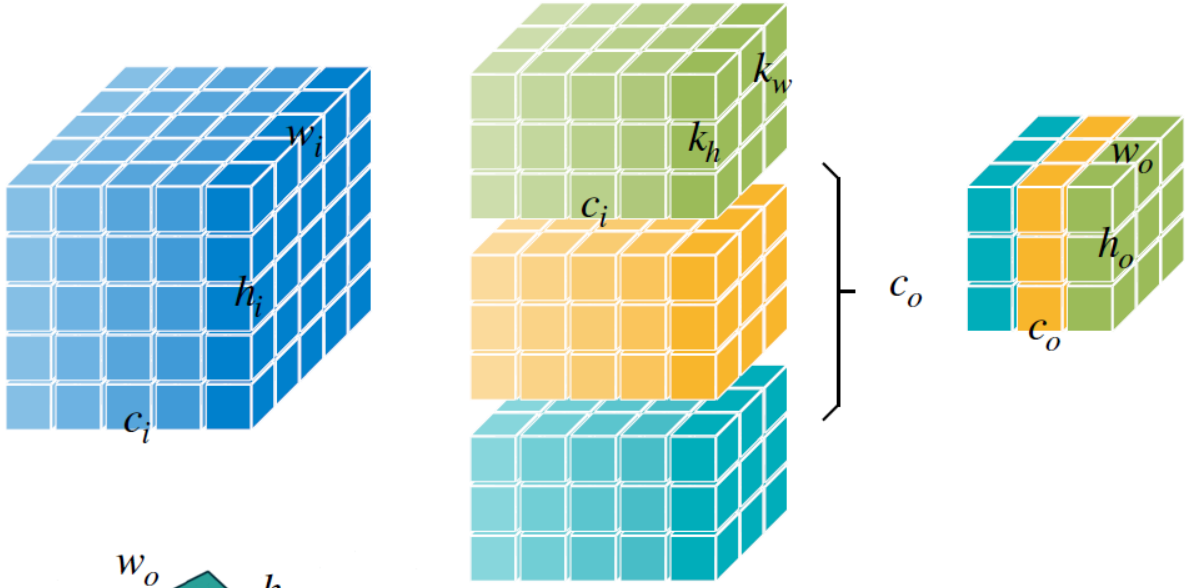
| Layer | #Parameters (bias is ignored) |
|---|---|
| Linear Layer | $c_o \cdot c_i$ |
| Convolution | |
| Grouped Convolution | |
| Depthwise Convolution | |




| Notations | |
|---|---|
| $n$ | Batch Size |
| $c_i$ | Input Channels |
| $c_o$ | Output Channels |
| $h_i$, $h_o$ | Input/Output Height |
| $w_i$, $w_o$ | Input/Output Width |
| $k_h$, $k_w$ | Kernel Height/Width |
| $g$ | Groups |

# Number of Parameters (#Parameters)

| Layer | #Parameters (bias is ignored) |
|---|---|
| Linear Layer | $c_o \cdot c_i$ |
| Convolution | $c_o \cdot c_i \cdot k_h \cdot k_w$ |
| Grouped Convolution | |
| Depthwise Convolution | |



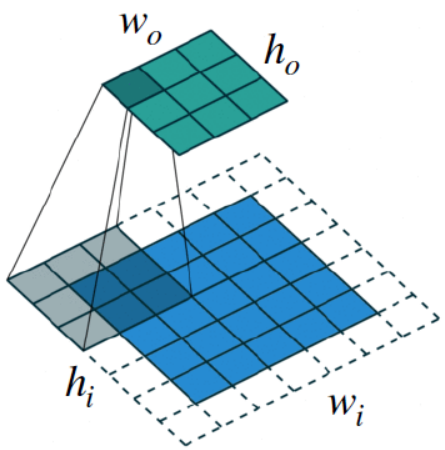




Image source: 1

| Notations | |
|---|---|
| $n$ | Batch Size |
| $c_i$ | Input Channels |
| $c_o$ | Output Channels |
| $h_i$, $h_o$ | Input/Output Height |
| $w_i$, $w_o$ | Input/Output Width |
| $k_h$, $k_w$ | Kernel Height/Width |
| $g$ | Groups |

# Number of Parameters (#Parameters)

| Layer | #Parameters (bias is ignored) |
|---|---|
| **Linear Layer** | $c_o \cdot c_i$ |
| **Convolution** | $c_o \cdot c_i \cdot k_h \cdot k_w$ |
| **Grouped Convolution** | $c_o/g \cdot c_i/g \cdot k_h \cdot k_w \cdot g$ <br> $= c_o \cdot c_i \cdot k_h \cdot k_w / g$ |
| **Depthwise Convolution** | |

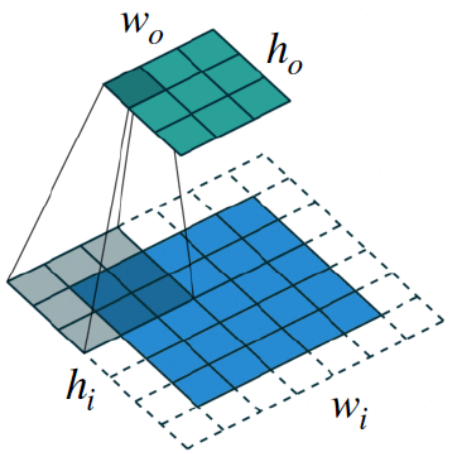
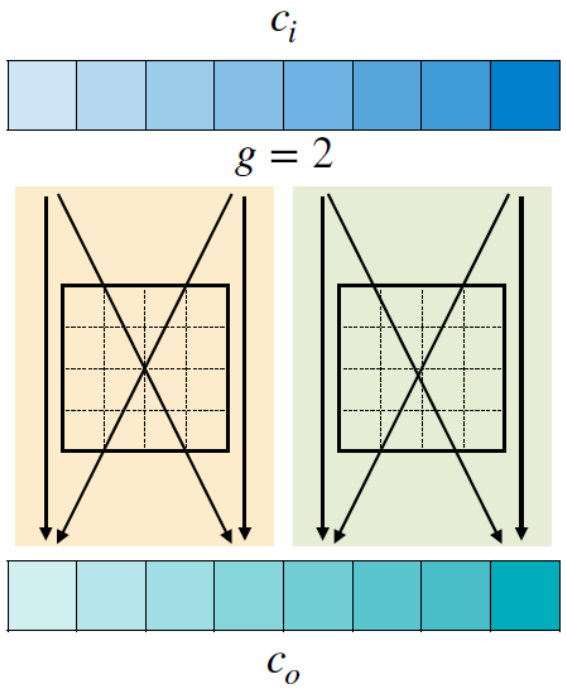


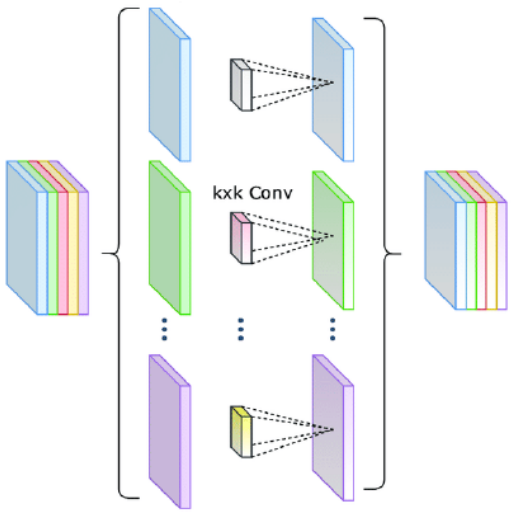

Image source: 1

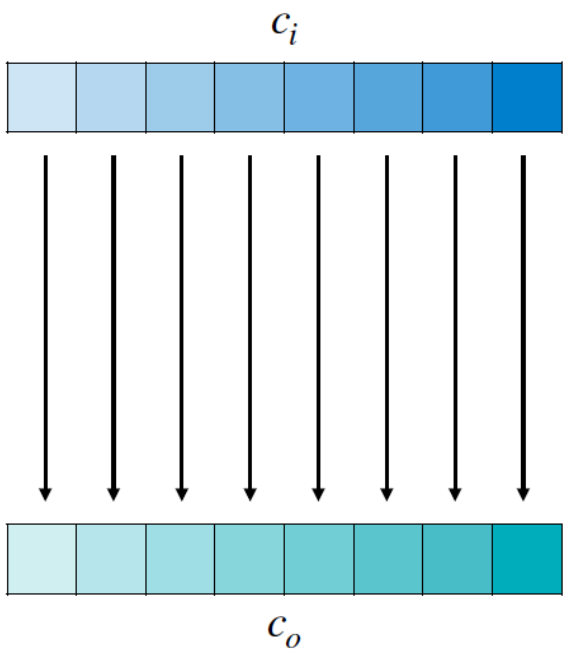



| Notations | |
|---|---|
| $n$ | Batch Size |
| $c_i$ | Input Channels |
| $c_o$ | Output Channels |
| $h_i, h_o$ | Input/Output Height |
| $w_i, w_o$ | Input/Output Width |
| $k_h, k_w$ | Kernel Height/Width |
| $g$ | Groups |

# Number of Parameters (#Parameters)

| Layer | #Parameters (bias is ignored) |
|---|---|
| **Linear Layer** | $c_o \cdot c_i$ |
| **Convolution** | $c_o \cdot c_i \cdot k_h \cdot k_w$ |
| **Grouped Convolution** | $c_o/g \cdot c_i/g \cdot k_h \cdot k_w \cdot g$ $= c_o \cdot c_i \cdot k_h \cdot k_w/g$ |
| **Depthwise Convolution** | $c_o \cdot k_h \cdot k_w$ |



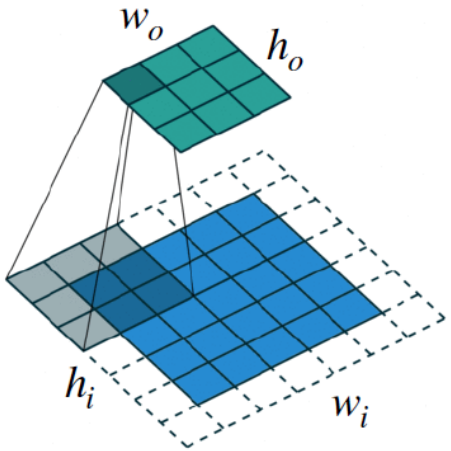




Image source: 1

| Notations | |
|---|---|
| $n$ | Batch Size |
| $c_i$ | Input Channels |
| $c_o$ | Output Channels |
| $h_i$, $h_o$ | Input/Output Height |
| $w_i$, $w_o$ | Input/Output Width |
| $k_h$, $k_w$ | Kernel Height/Width |
| $g$ | Groups |

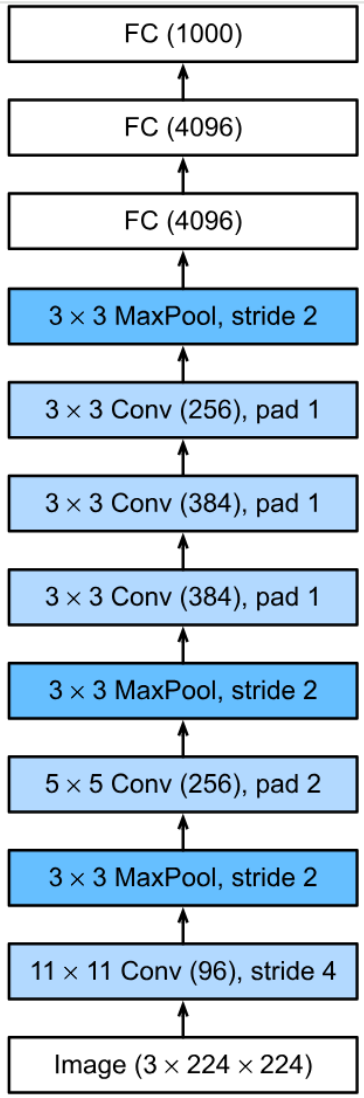
# Learnable Parameters (Trainable) in CNN

```
# Define the CNN model
model = keras.Sequential([
    layers.Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)),
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.Flatten(),
    layers.Dense(128, activation='relu'),
    layers.Dense(10, activation='softmax')  # Output layer for 10 classes
])
```

# Learnable Parameters (Trainable) in CNN

| Layer | Type | Input Shape | Output Shape | Kernel Size | Filters/Units | Parameters |
|---|---|---|---|---|---|---|
| Conv2D-1 | Conv | (28, 28, 1) | (26, 26, 32) | (3×3) | 32 | 320 |
| Conv2D-2 | Conv | (26, 26, 32) | (24, 24, 64) | (3×3) | 64 | 18,496 |
| Flatten | Reshape | (24, 24, 64) | (36,864) | — | — | 0 |
| FCN-1 | FC | (36,864) | (128) | — | 128 | 4,718,720 |
| Output | FC | (128) | (10) | — | 10 | 1,290 |
| Total | — | — | — | — | — | 4,738,826 |

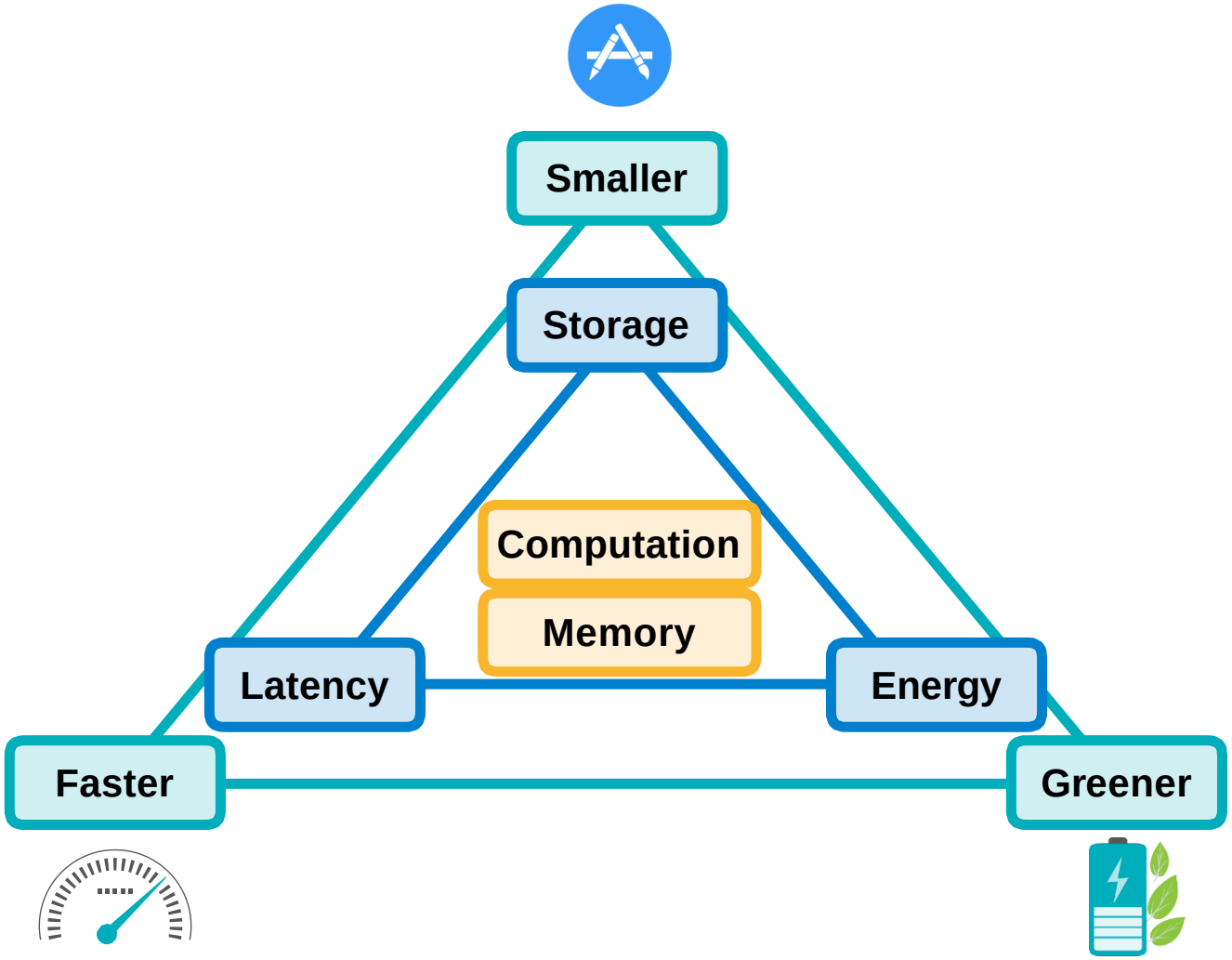
# AlexNet: #Parameters




| Layer | Type | Input Shape (C × H × W) | Output Shape (C × H × W) | Kernel Size | Filters | Parameters |
|---|---|---|---|---|---|---|
| Conv1 | Conv | (3 × 224 × 224) | (96 × 55 × 55) | (11 × 11) | 96 | 34,944 |
| Conv2 | Conv | (96 × 27 × 27) | (256 × 27 × 27) | (5 × 5) | 256 | 614,656 |
| Conv3 | Conv | (256 × 13 × 13) | (384 × 13 × 13) | (3 × 3) | 384 | 885,120 |
| Conv4 | Conv | (384 × 13 × 13) | (384 × 13 × 13) | (3 × 3) | 384 | 1,327,488 |
| Conv5 | Conv | (384 × 13 × 13) | (256 × 13 × 13) | (3 × 3) | 256 | 884,992 |
| FC1 | FC | (256 × 6 × 6) | (4096) | — | 4096 | 37,752,832 |
| FC2 | FC | (4096) | (4096) | — | 4096 | 16,781,312 |
| FC3 | FC | (4096) | (1000) | — | 1000 | 4,097,000 |
| Total | — | — | — | — | — | 60,965,872 |

61M in total

# Efficiency of Neural Networks




**Efficiency Metrics**

Memory-Related

#parameters

**model size**

total/peak #activations

Computation-Related

MAC

FLOP, FLOPS

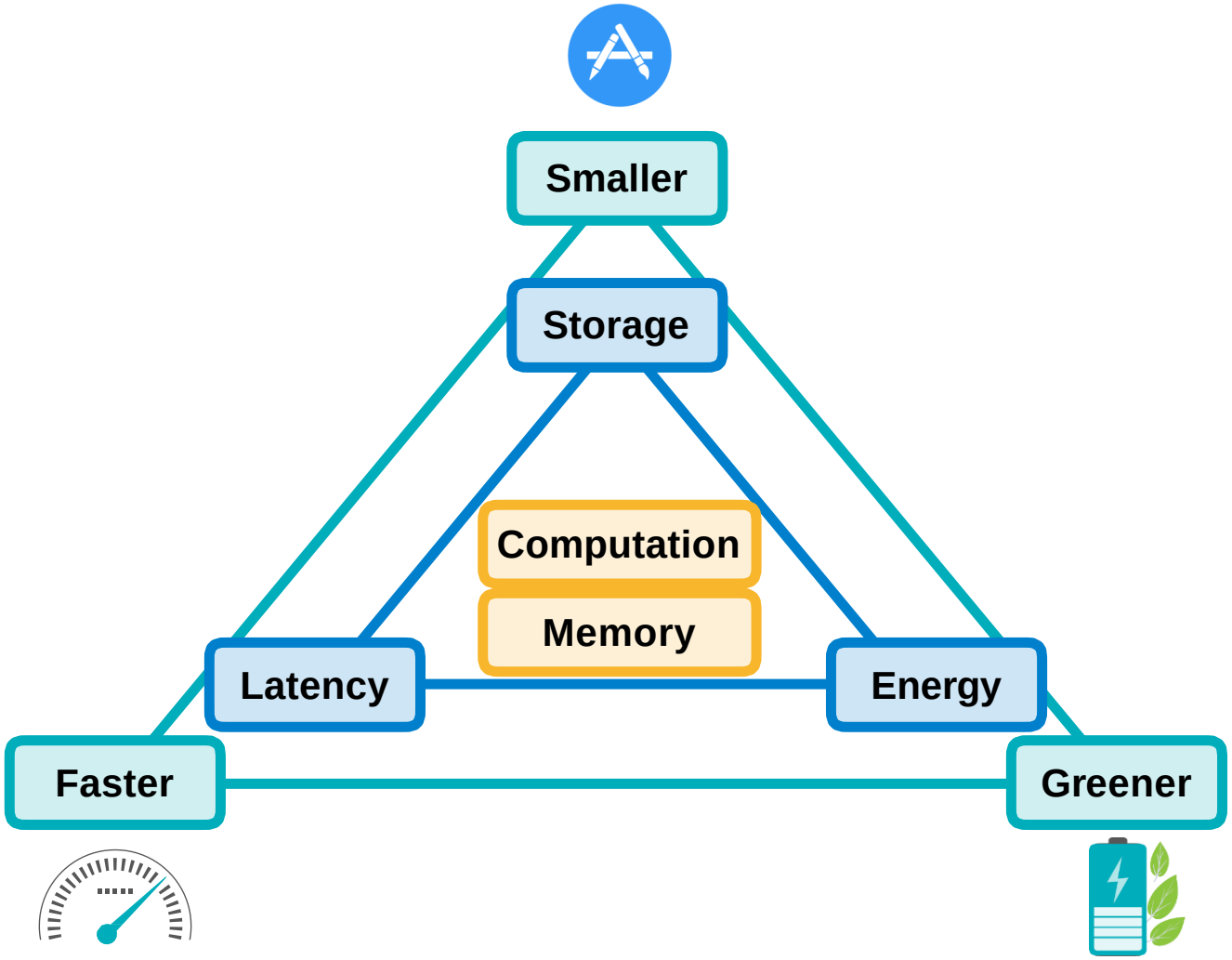
OP, OPS

# Model Size

- Model size measures the storage for the weights of the given neural network.
  - The common units for model size are: MB (megabyte), KB (kilobyte), bits.
- In general, if the whole neural network uses the same data type (*e.g.*, floating-point),
  - $Model\ \ Size = \#Parameters \cdot Bit\ Width$

- Example: AlexNet has 61M parameters.
  - If all weights are stored with 32-bit numbers, total storage will be about
    - 61M × 4 Bytes (32 bits) = 224 MB ($224 \times 10^6$ Bytes)
  - If all weights are stored with 8-bit numbers, total storage will be about
    - 61M × 1 Byte (8 bits) = 61 MB

# Efficiency of Neural Networks




**Efficiency Metrics**

Memory-Related

#parameters

model size

total/peak #activations

Computation-Related

MAC

FLOP, FLOPS

OP, OPS

# AlexNet: #Activations

| AlexNet | C × H × W | |
|---|---|---|
| Image (3×224×224) | 3×224×224 | =150,528 |
| 11×11 Conv, channel 96, stride 4, pad 2 | 96×55×55 | =290,400 |
| 3×3 MaxPool, stride 2 | 96×27×27 | =69,984 |
| 5×5 Conv, channel 256, pad 2, groups 2 | 256×27×27 | =186,624 |
| 3×3 MaxPool, stride 2 | 256×13×13 | =43,264 |
| 3×3 Conv, channel 384, pad 1 | 384×13×13 | =64,896 |
| 3×3 Conv, channel 384, pad 1, groups 2 | 384×13×13 | =64,896 |
| 3×3 Conv, channel 256, pad 1, groups 2 | 256×13×13 | =43,264 |
| 3×3 MaxPool, stride 2 | 256×6×6 | =9,216 |
| Linear, channel 4096 | 4096 | =4,096 |
| Linear, channel 4096 | 4096 | =4,096 |
| Linear, channel 1000 | 1000 | =1,000 |

**Total #Activation: 932,264**

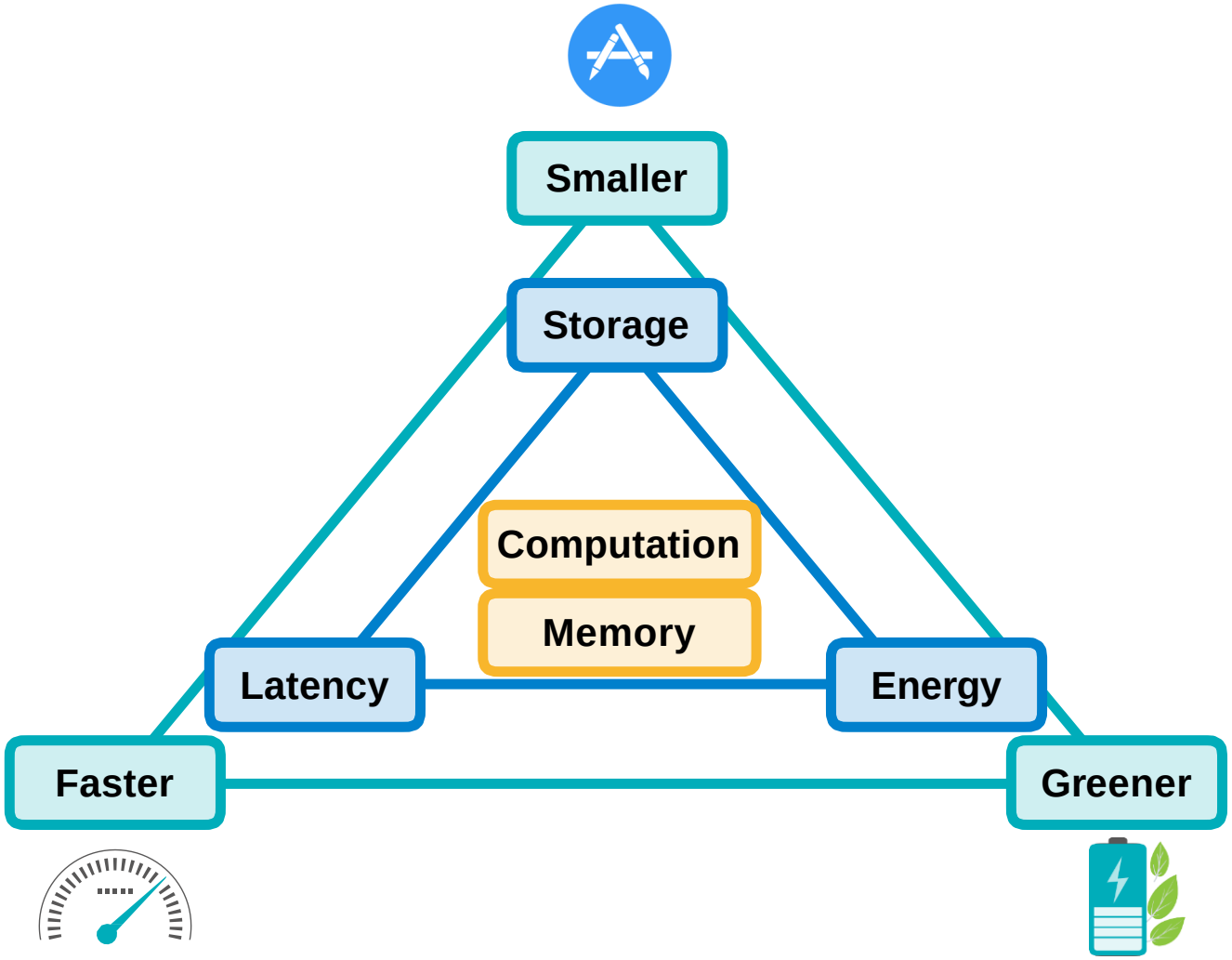
**Peak #Activation:**

**≈ #input activation + #output activation**

**= 150,528 + 290,400 = 440,928**

ImageNet Classification with Deep Conv Neural Networks [Krizhevsky *et al.*, NeurIPS 2012]

# Efficiency of Neural Networks




**Efficiency Metrics**

Memory-Related

#parameters

model size

total/peak #activations

Computation-Related

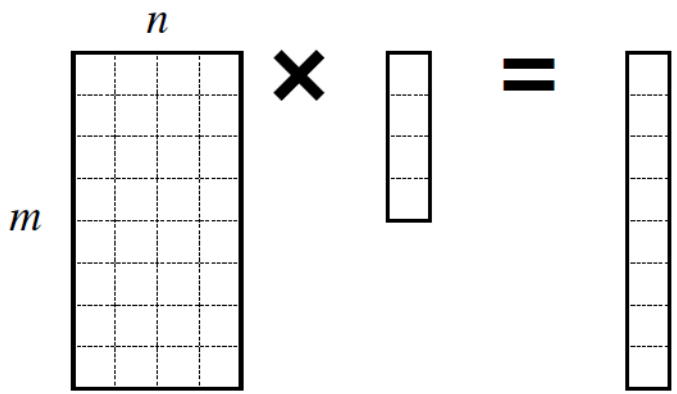
**MAC**

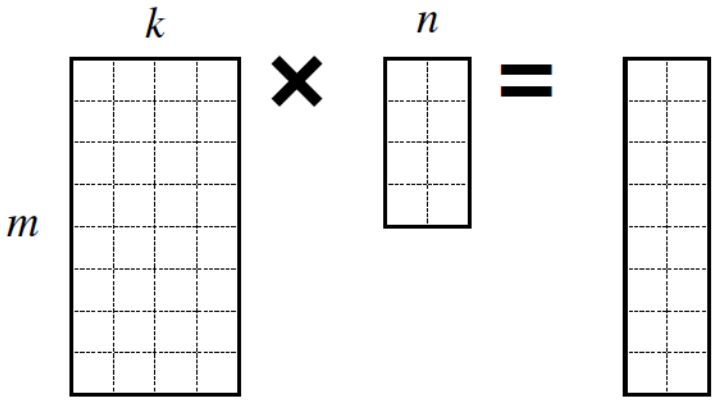
FLOP, FLOPS

OP, OPS

# Number of Multiply-Accumulate Operations

- Multiply-Accumulate operation (MAC)
  - $a \leftarrow a + b \cdot c$
- Matrix-Vector Multiplication (MV)
  - $MACs = m \cdot n$
- General Matrix-Matrix Multiplication (GEMM)
  - $MACs = m \cdot n \cdot k$

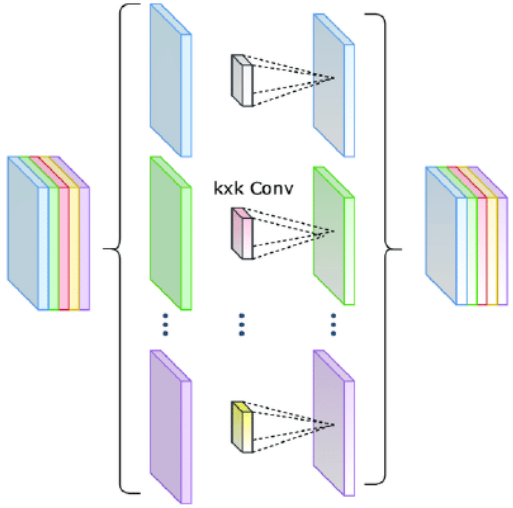


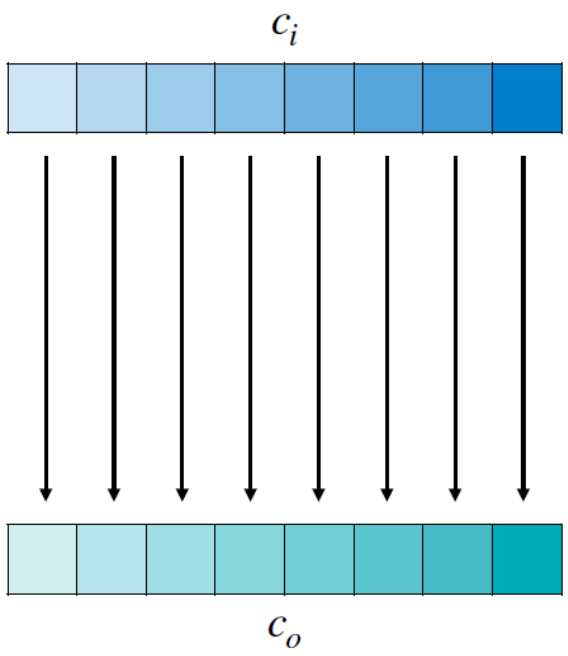



# Number of Multiply-Accumulate Operations

| Layer | MACs (batch size n=1) |
|---|---|
| **Linear Layer** | $c_o \cdot c_i$ |
| **Convolution** | $c_i \cdot k_h \cdot k_w \cdot h_o \cdot w_o \cdot c_o$ |
| **Grouped Convolution** | $c_i/g \cdot k_h \cdot k_w \cdot h_o \cdot w_o \cdot c_o$ |
| **Depthwise Convolution** | $k_h \cdot k_w \cdot h_o \cdot w_o \cdot c_o$ |

* bias is ignored



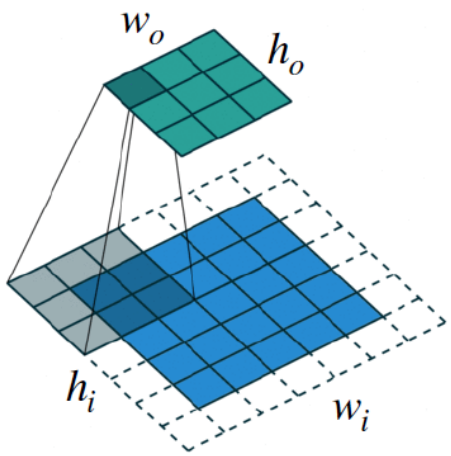




Image source: 1

| Notations | |
|---|---|
| $n$ | Batch Size |
| $c_i$ | Input Channels |
| $c_o$ | Output Channels |
| $h_i$, $h_o$ | Input/Output Height |
| $w_i$, $w_o$ | Input/Output Width |
| $k_h$, $k_w$ | Kernel Height/Width |
| $g$ | Groups |

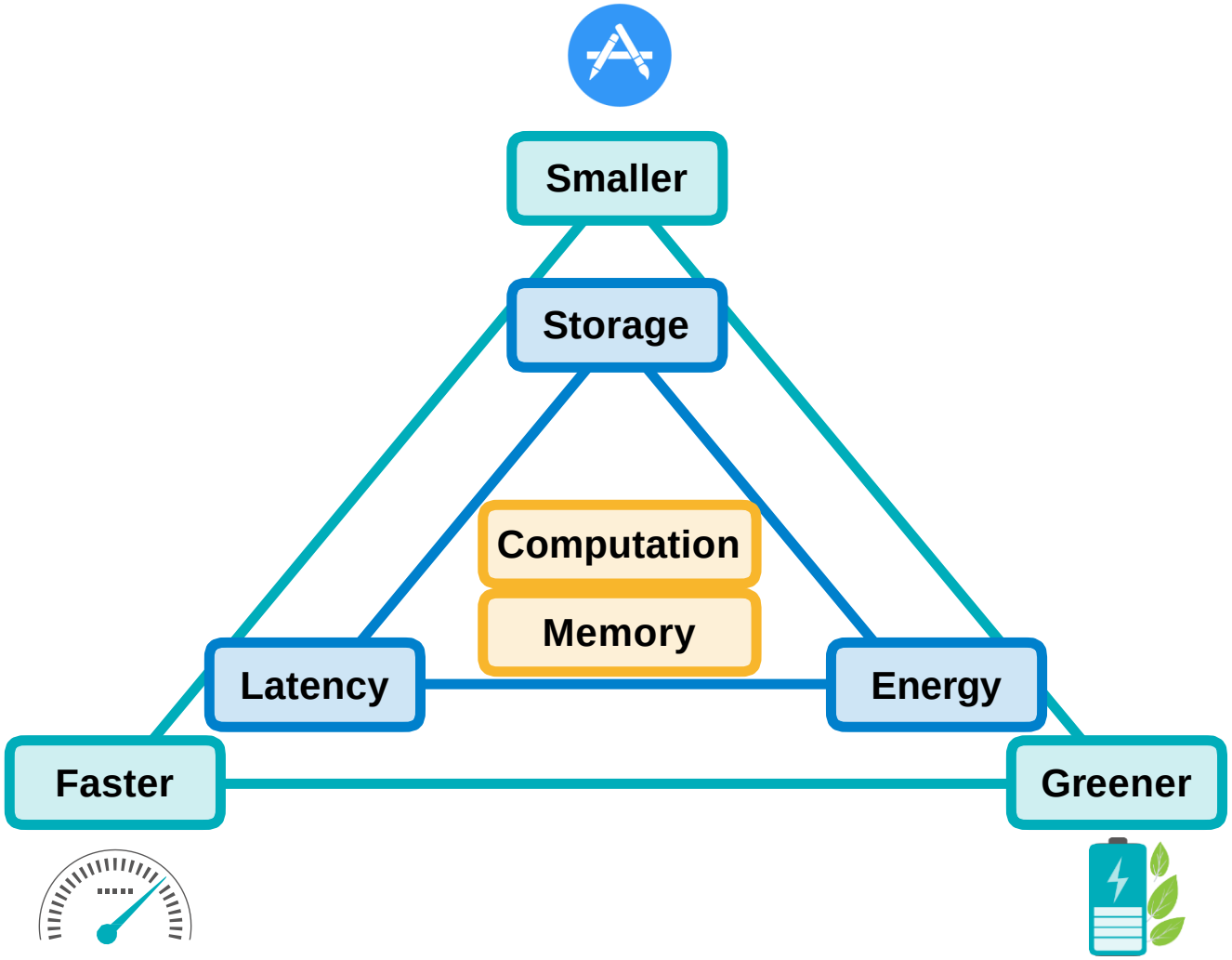
# AlexNet: #MACs

| AlexNet | C × H × W | MACs |
|---|---|---|
| Image (3×224×224) | 3×224×224 | |
| 11×11 Conv, channel 96, stride 4, pad 2 | 96×55×55 | 96×3×11×11×55×55<br>= 105,415,200 |
| 3×3 MaxPool, stride 2 | 96×27×27 | |
| 5×5 Conv, channel 256, pad 2, groups 2 | 256×27×27 | 256×96×5×5×27×27 / 2<br>= 223,948,800 |
| 3×3 MaxPool, stride 2 | 256×13×13 | |
| 3×3 Conv, channel 384, pad 1 | 384×13×13 | 384×256×3×3×13×13<br>= 149,520,384 |
| 3×3 Conv, channel 384, pad 1, groups 2 | 384×13×13 | 384×384×3×3×13×13 / 2<br>= 112,140,288 |
| 3×3 Conv, channel 256, pad 1, groups 2 | 256×13×13 | 256[illegible]×3×3×13×13 / 2<br>= 74,[illegible] |
| 3×3 MaxPool, stride 2 | 256×6×6 | |
| Linear, channel 4096 | 4096 | 4096×(256×6×6)<br>= 37,748,736 |
| Linear, channel 4096 | 4096 | 4096×4096<br>= 16,777,216 |
| Linear, channel 1000 | 1000 | 1000×4096<br>= 4,096,000 |

**724M in total !**

ImageNet Classification with Deep Conv Neural Networks [Krizhevsky *et al.*, NeurIPS 2012]

# Efficiency of Neural Networks




**Efficiency Metrics**

**Memory-Related**

#parameters

model size

total/peak #activations

**Computation-Related**

MAC

**FLOP, FLOPS**

**OP, OPS**

Image source: 1

# Number of Floating Point Operations (FLOP)

- A multiply is a floating point operation
- An add is a floating point operation
- ***One*** multiply-accumulate operation is ***two*** floating point operations (FLOP)
  - Example: AlexNet has 724M MACs, total number of floating point operations will be
    - 724M × 2 = 1.4G FLOPs

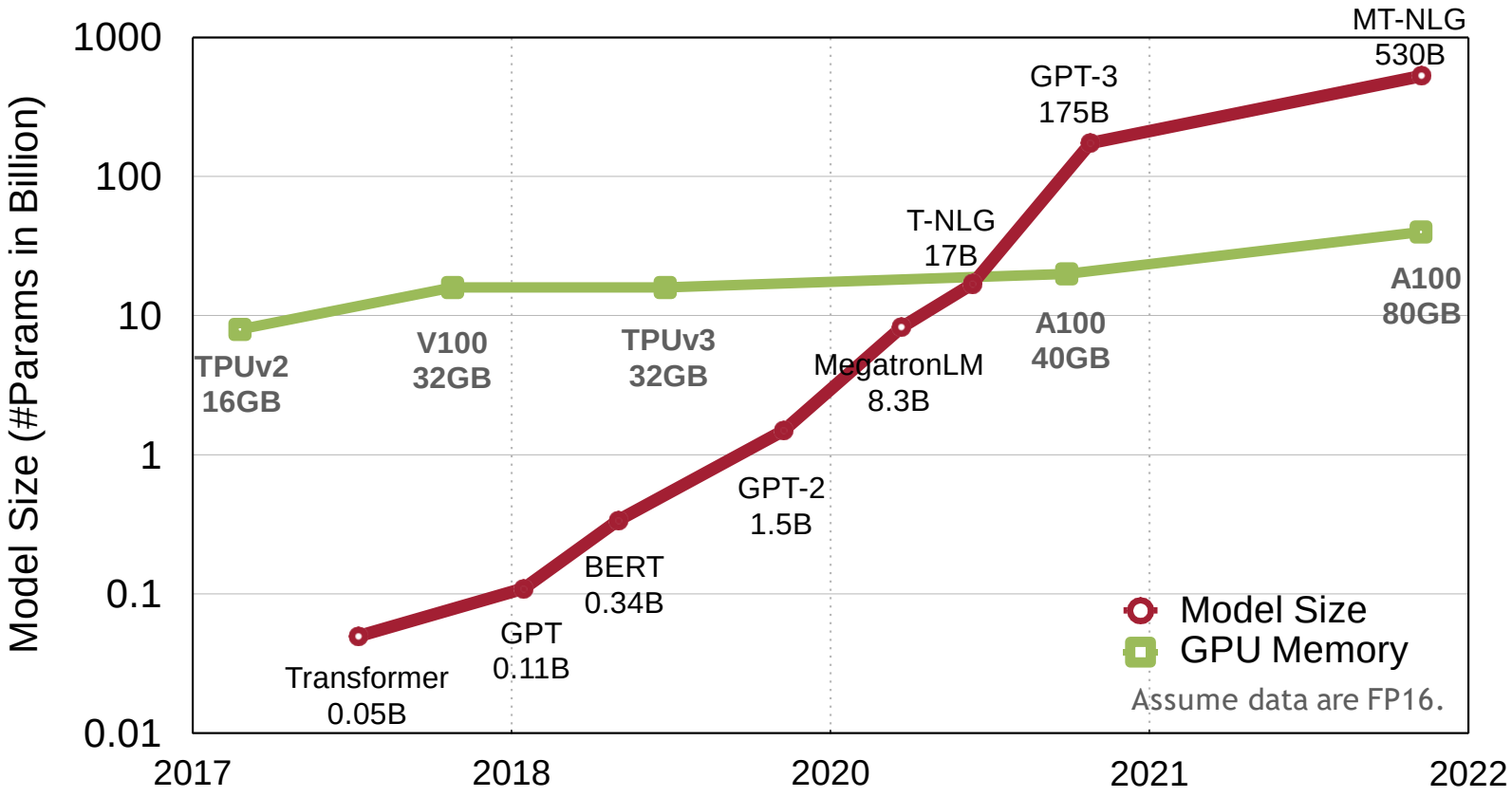
- Floating Point Operation Per Second (FLOPS)
  - $FLOPS = \frac{FLOPs}{second}$

# Number of Operations (OP)

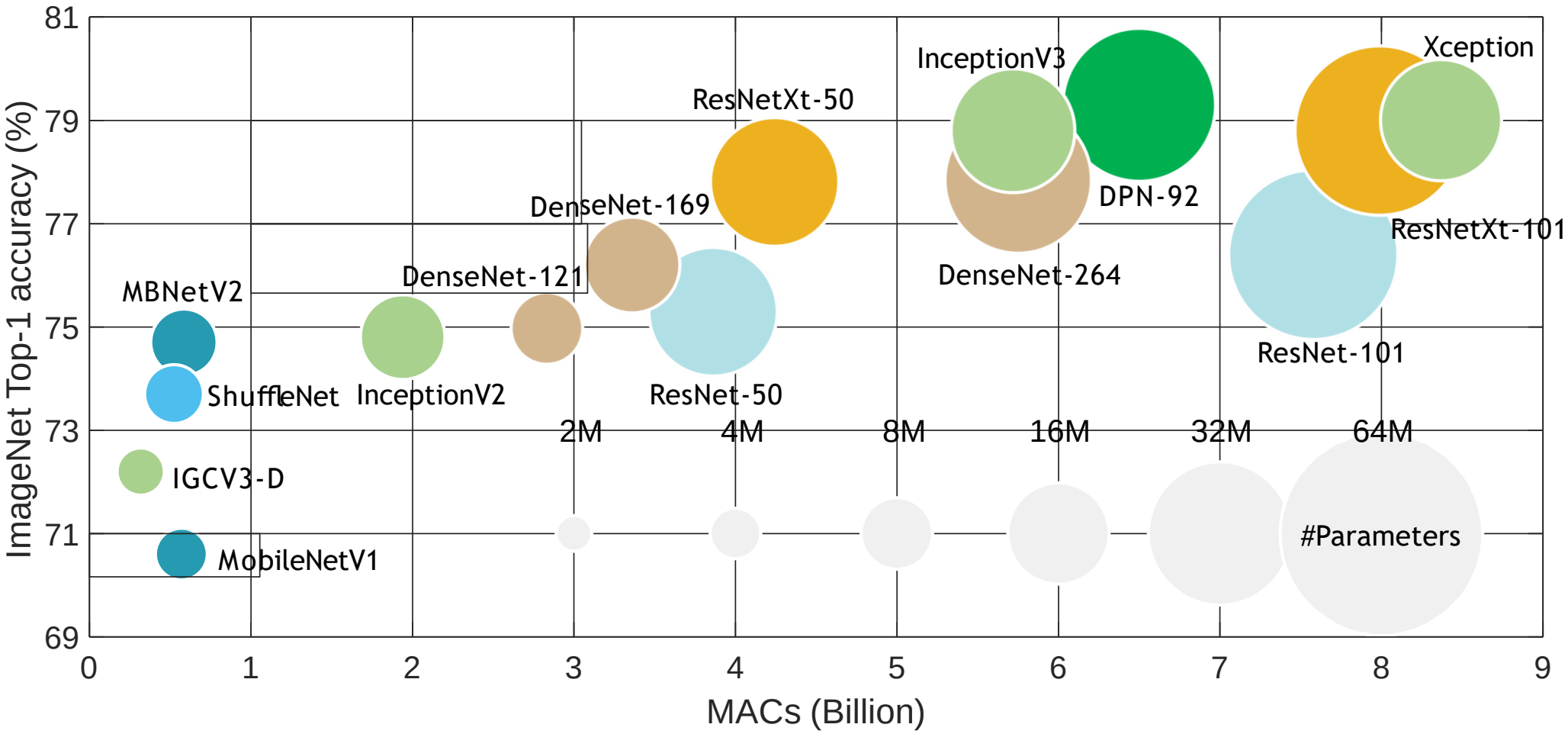
- Activations/weights in neural network computing are not always floating point.
- To generalize, number of operations is used to measure the computation amount.
  - Example: AlexNet has 724M MACs, total number of floating point operations will be
    - 724M × 2 = 1.4G OPs

- Similarly, Operation Per Second (OPS)
  - $OPS = \frac{OPs}{second}$

# Comparison of Model Size



# Comparison of MACs




Model Compression and Hardware Acceleration for Neural Networks: A Comprehensive Survey [Deng *et al.*, IEEE 2020]

Source: https://premioinc.com